Plauthium Technical Specification

Abstract

Plauthium is an application-specific post-quantum layer-1 for tokenized verifiable computation. Verified Fresh Work is the sole source of chain weight, reward rights and execution rights. It is a chain-bound exact product of two 512×512 signed-INT8 matrices with INT32 accumulation. Consensus validators regenerate the operands from the block context, hash the complete supplied result, and verify correctness in $O(n^2)$ work via six Fiat–Shamir Freivalds rounds over $\mathbb{F}_p$, $p = 2^{61} - 1$, giving a static false-product error of at most $p^{-6} \approx 2^{-366}$. Three deterministic diagnostic profiles—signed outlier arithmetic, nonnegative graph kernels, and correlated accumulator stress—rotate on a strict modulo-three schedule to form an open compute-validation kit. Commercial computation executes in the Scientific Virtual Machine (SciVM), is certified by a mixed-field Circle STARK with challenges in $\mathbb{F}_{p^8} = \mathbb{F}_p[X]/(X^8 + X + 9)$, and carries zero consensus weight. One Fresh Work evaluation feeds a seal lottery, which carries chain weight, and a share lottery, which carries reward and execution rights. Execution slots are allocated by the sequential D'Hondt method over discrete ShareIDs. Every consensus node reconstructs the rate-1/8 Reed–Solomon data layer. The transparent public UTXO and protocol-account state machine conserves a fixed supply of 1.5×10^{15} atoms, that is 15,000,000 PLAU, with no financial shielded notes or nullifiers. Primary issuance releases 10% linearly over 1,051,200 blocks into the `InitialProtocolPool`, whose gross outflow is bounded by a rolling 720-block `InitialValve` at 30,000 PLAU, while 90% releases to miners via an exact integer Q64 exponential decay. The Base Fee moves irretrievably to the locked reserve at admission and is recycled into independent integer fee lots; under the uniform admission-conditioned fairness bound with $\bar{\alpha}_\varepsilon \beta^{L+1} < 1$, self-dealing loses at least $(1 - \bar{\alpha}_\varepsilon \beta^{L+1})F$ in expectation. Signatures, envelopes and commitments use ML-DSA-87, SLH-DSA-SHAKE-256s, ML-KEM-1024 and SHA3-512 from Genesis.

Table of Contents

Abstract **1**

1. Introduction and the Plauthium Model **2**
- 1.1. Design Thesis . . . 2
- 1.2. Architecture . . . 2
- 1.3. Normative Parameter Core . . . 3
- 1.4. Encoding Conventions . . . 3
- 1.5. Adversary Model . . . 4

2. Post-Quantum Cryptography Foundation **4**
- 2.1. Genesis Suite . . . 4
- 2.2. Signatures and Authorization . . . 5
- 2.3. KEM–DEM Envelopes . . . 5
- 2.4. Digest Length against Grover and BHT . . . 5
- 2.5. Post-Quantum Soundness of the Proof Layer . . . 6

3. Module 1 — Compute Primitive and Circle STARK **6**
- 3.1. Canonical Tile . . . 6
- 3.2. Stochastic Verification . . . 6
- 3.3. Mixed-Field Circle STARK . . . 7
- 3.4. Error Budget and Tile Decomposition . . . 7
- 3.5. Prover Memory . . . 8
- 3.6. Scientific Virtual Machine (SciVM) and the Ozaki-II Profile . . . 8

4. Module 2 — Prover Data Availability Layer (P-DAL) **9**
- 4.1. Encoding and Commitment . . . 9
- 4.2. Data Availability Sampling . . . 9
- 4.3. Reconstruction Rule . . . 10

4.4. Fast-Path Certificate F1 . . . 10

5. Module 3 — Dual-Lottery Consensus and Anti-Grinding Fresh Work 10
5.1. Fresh Work Context and Three Diagnostic Generators . . . 10
5.2. Instance Generation and Identifiers . . . 11
5.3. Dual-Lottery and Six-Round Freivalds Verification . . . 11
5.4. Witness-Independent Lottery Identifiers . . . 12
5.5. Fork Choice . . . 12
5.6. Delay and Reorganization Bound . . . 12
5.7. Rewards and Deviation Bound . . . 13

6. Module 4 — Deterministic Scheduler and D'Hondt Allocation 13
6.1. ShareID-Indexed Weights . . . 13
6.2. Floor-Quota Defect . . . 14
6.3. Sequential D'Hondt Recursion . . . 14
6.4. Completeness and Proportionality . . . 14
6.5. Aggregation Invariance . . . 15
6.6. Leases and Phase Machine . . . 15
6.7. Ordering and Throughput . . . 16

7. Module 5 — Transparent State, Transactions and Tokenomics 17
7.1. State and Operation Set . . . 17
7.2. Public TWL Authorization and Conflicts . . . 17
7.3. Transparent Escrow Funding . . . 18
7.4. Commercial Trade-Secret Abstraction . . . 18
7.5. Atomic Transitions and Settlement . . . 18
7.6. Supply Invariant, Decoupled Issuance and InitialProtocolPool . . . 19
7.7. Fee Schedule and Anti-Spam Base Fee . . . 20
7.8. Recycling and Exact Reserve Dynamics . . . 21
7.9. Self-Dealing Theorem . . . 21
7.10. Resource Limits and Transparent L2 Slots . . . 22

References 22

1. Introduction and the Plauthium Model

1.1. Design Thesis

Chain weight, economic rights and commercial payload are produced by disjoint maps. Fresh Work [24]–[28] enters only the first two, and the payload enters neither (1.1). A buyer of computation therefore cannot purchase weight, and a miner cannot convert payments into fork-choice influence.

$$\mathcal{W} = \mathcal{W}\big(\mathrm{seals}(\mathcal{C})\big), \qquad \mathsf{Rights} = \mathsf{Rights}\big(\mathcal{S}_{\mathrm{mature}}\big), \qquad \mathrm{seals}(\mathcal{C}) = \mathrm{seals}(\mathcal{C}') \;\Rightarrow\; \mathcal{W}(\mathcal{C}) = \mathcal{W}(\mathcal{C}') \tag{1.1}$$

Legend:

- $\mathcal{C}, \mathcal{C}'$ — chains that may differ in commercial results, fees and bonds; $\mathrm{seals}(\mathcal{C})$ — the multiset of Fresh Work seals of $\mathcal{C}$;
- $\mathcal{W}$ — the cumulative chain weight (Section 5.5);
- Rights — reward and execution entitlements; $\mathcal{S}_{\mathrm{mature}}$ — the mature Fresh Work shares.

1.2. Architecture

The protocol is a post-quantum foundation PQC composed with five modules $\mathsf{M}_1, \ldots, \mathsf{M}_5$. Three flows connect them (1.2): the Fresh Work flow, the job flow and the fee flow.

$$\begin{aligned} & x \xrightarrow{\mathsf{M}_1,\ \mathsf{M}_3} (C,\ \mathit{outputRoot}) \xrightarrow{\mathrm{Freivalds}_6} (\vartheta_B,\ \vartheta_S) \longmapsto (\mathcal{W},\ \mathsf{Rights}) \\ & \mathsf{Escrow} \xrightarrow{\mathsf{M}_4} \mathsf{Lease} \xrightarrow{\mathsf{M}_1,\ \mathsf{M}_2} (Z,\ \pi,\ \text{available data}) \xrightarrow{\mathsf{M}_5} \Sigma' \\ & F^{\mathrm{base}} \longrightarrow R_l \xrightarrow{\text{maturation}} R_a \longrightarrow \text{Fresh Work rewards} \end{aligned} \tag{1.2}$$

Legend:

- M_1 — compute primitive, SciVM and Circle STARK; M_2 — prover data availability layer; M_3 — Dual-Lottery consensus and diagnostic matrix generators; M_4 — deterministic scheduler; M_5 — transparent state, transactions and tokenomics;
- x — the Fresh Work context; C, *outputRoot* — the canonical INT32 matrix product and its commitment; ϑ_B, ϑ_S — the lottery values;
- Z, π — a commercial result and its STARK proof; Σ' — the settled state;
- F^{base} — the Base Fee; R_l, R_a — the locked and the available fee reserve.

1.3. Normative Parameter Core

The values in Table 1 are normative for all modules. Every other quantity is a deployment parameter of the versioned release manifest.

Parameter	Value	Defined in
Base field	$p = 2^{61} - 1 = 2\,305\,843\,009\,213\,693\,951$	3.1
Challenge field	$\mathbb{F}_{p^8} = \mathbb{F}_p[X]/(X^8 + X + 9)$	3.3
Native tile	$n = 512$; INT8 inputs in $[-127, 127]$; INT32 result	3.1
SciVM profile	deterministic Ozaki-II residue splitting over native INT8 tiles	3.6
Freivalds rounds	$r_F = 6$; static error $\leq p^{-6} \approx 2^{-366}$	3.2
Prover memory	peak ≤ 19.5 GiB	3.5
Signatures	ML-DSA-87 (FIPS 204); treasury: ML-DSA-87 $\wedge$ SLH-DSA-SHAKE-256s (FIPS 205)	2.2
Envelopes	ML-KEM-1024 (FIPS 203), HKDF-SHA-512, AES-256-GCM	2.3
Hash, XOF, PRF	SHA3-512, SHAKE256, KMAC256	2.4
Fresh Work profiles	Outlier, arithmetic graph, correlated accumulator stress; schedule $(h - h_{\text{G}} - 1) \bmod 3$, generator IDs $3100 + j$	5.1
Result witness	1,048,576 B row-major signed INT32 little-endian; full-output SHA3-512 commitment	5.2
P-DAL	Reed–Solomon, rate 1/8; full reconstruction by consensus nodes	4.1
Nominal rates	$T_B = 120$ s; $\lambda_S = 1$ s^{-1}; target share/seal ratio 120	5.3
Scheduler	sequential D'Hondt, quotient $w_s/(a_s + 1)$ over ShareIDs	6.3
Supply	$M = 1.5 \times 10^{15}$ atoms = 15,000,000 PLAU	7.6
Initial Protocol Pool	$I = 10\%M$ released linearly over $N_I = 1{,}051{,}200$ blocks into `InitialProtocolPool`	7.6
InitialValve	Rolling $W = 720$ blocks; gross outflow $\leq I/50 = 30{,}000$ PLAU per window	7.6
Miner Release	$R = 90\%M$; integer Q64 exponential decay ($Q = 2^{64}$, $a = 18\,446\,731\,910\,177\,940\,463$)	7.6
Base Fee	irretrievable transfer to R_l at `JOB_ADMIT_AND_LEASE`; recycled, never burned	7.7
Commercial weight	0	1.1

Table 1: Normative parameter core.

1.4. Encoding Conventions

Every hashed object is domain-separated and injectively encoded, and every consensus quantity is an integer, (1.3)–(1.4). Fractional expressions such as $2^{512}/\Theta_B$ are analytic notation for integer computations.

$$H_{512} := \mathsf{SHA3\text{-}512}, \qquad \mathit{digest} = H_{512}\big(\mathsf{Enc}(\texttt{LABEL},\ f_1, \ldots, f_k)\big), \qquad \mathsf{Enc}\ \text{injective} \tag{1.3}$$

$$\frac{a}{b} < \frac{c}{d} \iff a\,d < c\,b \quad (a, b, c, d \in \mathbb{Z},\ b, d > 0) \tag{1.4}$$

Legend:

- H_{512} — SHA3-512; *digest* — any 512-bit commitment, identifier or lottery value, read as a big-endian integer;
- Enc — the typed, length-prefixed encoding with fixed field order that rejects non-canonical forms;

- `LABEL` — a canonical domain label, distinct for every use; $f_1, \ldots, f_k$ — the encoded fields;
- (1.4) — the exact comparison rule for ratios; no consensus rule uses floating point.

1.5. Adversary Model

Every theorem in Sections 2 to 7 holds in the model of Table 2.

Assumption	Formal statement	Used in
A1 — Fresh Work	$\mathrm{cost}_{\mathcal{A}}(\text{attempt on fresh } x) \gtrsim \mathrm{cost}(\mathsf{Eval}_{\mathrm{FW}})$; residual speed-ups, including quantum ones, enter α	5.4
A2 — Bounded delay	$t_{\text{discovery}\to\text{validation}} \le \Delta_{\text{eff}}$, as a bound and not a mean	5.6
A3 — Honest advantage	$u = (1-\alpha)\exp(-\Delta_{\text{eff}}/T_B) > 1/2$ at constant rates; α is the adversarial share of effective seal capacity	5.6, 6.7
A4 — Proof profile	$\varepsilon_{\mathrm{STARK}} + \varepsilon_{\mathrm{commit}} \le 2^{-128}$ over the complete versioned compute profile in the QROM	2.5, 3.4
A5 — Fairness event	$\mathcal{F}_\delta$: every honest group of share φ receives at least $(1-\delta_f)\varphi E_e$ [5], [6]	5.7, 7.9
A6 — Static sampling	The withheld set is fixed and samples are drawn without replacement; no selective responses, query observation or isolation	4.2
A7 — Quantum interface	The adversary computes quantumly; honest signing and decapsulation interfaces are classical	2.3, 2.4

Table 2: Model assumptions.

2. Post-Quantum Cryptography Foundation

2.1. Genesis Suite

The Genesis suite consists of final NIST standards [39]–[47]. No component's security reduces to factoring or discrete logarithms (2.1)–(2.2). Suites change only through a versioned consensus rule, which excludes downgrade. Parameter sets and object sizes are given in Table 3.

Function	Scheme, NIST category	Sizes
Operation signatures	ML-DSA-87, category 5	public key 2592; signature 4627
Treasury co-signature	SLH-DSA-SHAKE-256s, category 5	public key 64; signature 29,792
Envelope key establishment	ML-KEM-1024, category 5	encapsulation key 1568; ciphertext 1568; secret 32
Content encryption	AES-256-GCM	key 32; nonce 12; tag 16
Commitments, identifiers, lotteries	SHA3-512	digest 64
Expansion and keyed derivation	SHAKE256, KMAC256	PRF output 64

Table 3: Genesis parameter sets and sizes in bytes.

$$\begin{aligned}\mathsf{Suite}_{\mathrm{Gen}} = \big(&\mathsf{ML\text{-}DSA\text{-}87}_{[\text{FIPS 204}]},\ \mathsf{SLH\text{-}DSA\text{-}SHAKE\text{-}256s}_{[\text{FIPS 205}]},\\ &\mathsf{ML\text{-}KEM\text{-}1024}_{[\text{FIPS 203}]},\ \mathsf{AES\text{-}256\text{-}GCM}_{[\text{SP 800-38D}]},\ \mathsf{HKDF\text{-}SHA\text{-}512}_{[\text{RFC 5869}]},\\ &\mathsf{SHA3\text{-}512},\ \mathsf{SHAKE256}_{[\text{FIPS 202}]},\ \mathsf{KMAC256}_{[\text{SP 800-185}]}\big)\end{aligned} \tag{2.1}$$

$$\begin{aligned}&\mathsf{Suite}_{\mathrm{Gen}} \cap \{\text{ECDSA, Schnorr, BLS, Pedersen, KZG, pairings, Groth16, DH}\} = \varnothing\\ &\mathit{obj.cryptoSuiteID} \in \mathcal{K}_{\mathrm{active}}(h)\end{aligned} \tag{2.2}$$

Legend:

- $\mathsf{Suite}_{\mathrm{Gen}}$ — the closed Genesis suite; subscripts name the defining standard;
- the excluded set — every primitive whose security reduces to factoring or discrete logarithms;
- *obj* — any signed, encrypted or committed protocol object; $\mathcal{K}_{\mathrm{active}}(h)$ — the set of suites active at height h, changed only by a versioned consensus rule.

2.2. Signatures and Authorization

Every public spend carries a pure-mode ML-DSA-87 signature over the complete effect digest, and the presented key must match the locking policy read from state (2.3)–(2.4). Initial protocol pool withdrawals and upgrade outputs require the conjunction with SLH-DSA-SHAKE-256s under independent keys (2.5); a disjunction is not admitted.

$$AuthMsg = \mathsf{Enc}(\texttt{PLAU/TX},\ TxID), \qquad \sigma \leftarrow \mathsf{ML\text{-}DSA\text{-}87.Sign}(sk,\ AuthMsg,\ ctx) \tag{2.3}$$

$$\mathsf{Accept}(tx) \iff \mathsf{ML\text{-}DSA\text{-}87.Verify}(pk,\ AuthMsg,\ \sigma,\ ctx) = 1 \ \wedge\ \mathsf{Match}(pk,\ lockingPolicy_{\mathrm{prev}}) = 1 \tag{2.4}$$

$$\begin{aligned} \mathsf{Accept}_{\mathrm{treasury}}(tx) \iff\ & \mathsf{ML\text{-}DSA\text{-}87.Verify}(pk_1,\ AuthMsg,\ \sigma_1,\ ctx) = 1 \\ & \wedge\ \mathsf{SLH\text{-}DSA\text{-}SHAKE\text{-}256s.Verify}(pk_2,\ AuthMsg,\ \sigma_2,\ ctx) = 1 \\ & \wedge\ \mathsf{Match}\big((pk_1,\ pk_2),\ lockingPolicy_{\mathrm{prev}}\big) = 1 \end{aligned} \tag{2.5}$$

Legend:

- tx — the transaction; $TxID$ — its effect identifier (7.5); $AuthMsg$ — the signed message;
- $ctx = \texttt{PLAU/TX}$ — the context string of FIPS 204 and FIPS 205;
- sk, pk — an ML-DSA-87 key pair; $\sigma, \sigma_1, \sigma_2$ — signatures;
- pk_1, pk_2 — independent ML-DSA-87 and SLH-DSA-SHAKE-256s keys;
- $lockingPolicy_{\mathrm{prev}}$ — the policy of the consumed output, read from state; Match — key-to-policy conformance.

2.3. KEM–DEM Envelopes

Confidential commercial payloads are sealed under ML-KEM-1024 with a fresh encapsulation per envelope, HKDF-SHA-512 and AES-256-GCM (2.6) [42], [45], [46]. The recipient key ek_R is authenticated by an ML-DSA-87 signature. Freshness excludes key–nonce reuse (2.7).

$$\begin{aligned} (kemCT,\ ss) &\leftarrow \mathsf{ML\text{-}KEM\text{-}1024.Encaps}(ek_R) \\ ectx &= \mathsf{Enc}(chainID,\ suiteID,\ payloadCommitment,\ outputIndex,\ recipientKeyEpoch,\ purpose) \\ prk &= \mathsf{HKDF\text{-}Extract}\big(H_{512}(\mathsf{Enc}(\texttt{ENVELOPE/SALT},\ ectx,\ kemCT)),\ ss\big) \\ k &= \mathsf{HKDF\text{-}Expand}\big(prk,\ \mathsf{Enc}(\texttt{ENVELOPE/KEY},\ ectx),\ 32\big), \qquad nonce \xleftarrow{\mathrm{R}} \{0,1\}^{96} \\ (ct,\ tag) &\leftarrow \mathsf{AES\text{-}256\text{-}GCM.Seal}\big(k,\ nonce,\ m,\ \mathsf{Enc}(ectx,\ kemCT,\ nonce)\big) \end{aligned} \tag{2.6}$$

$$i \neq j \ \Rightarrow\ (k_i,\ nonce_i) \neq (k_j,\ nonce_j) \tag{2.7}$$

Legend:

- ek_R — the authenticated encapsulation key of the recipient; $kemCT, ss$ — the KEM ciphertext and the 32-byte shared secret;
- $ectx$ — the envelope context, which binds purpose and payload commitment but not $TxID$ (no circular definition);
- prk, k — the HKDF pseudorandom key and the 32-byte AEAD key; HKDF is instantiated with SHA-512; m — the plaintext;
- ct, tag — the ciphertext and the 128-bit tag; $nonce$ — a fresh 96-bit value;
- i, j — envelope indices.

2.4. Digest Length against Grover and BHT

Grover search [48] inverts an n-bit hash with about $2^{n/2}$ queries, and the Brassard–Høyer–Tapp algorithm [49] finds a collision with about $2^{n/3}$ queries (2.8). Only digests of at least 384 bits keep the quantum collision cost at the 128-bit target, so SHA3-512 is used for every Merkle root, commitment, transaction identifier, work identifier and lottery value (2.9). Inside the AIR, a digest is carried as sixteen range-checked 32-bit limbs and is never reduced modulo p (2.10).

$$Q_{\mathrm{pre}}(n) \sim 2^{n/2}, \qquad Q_{\mathrm{col}}(n) \sim 2^{n/3} \tag{2.8}$$

$$Q_{\mathrm{col}}(256) \approx 2^{85.3} < 2^{128}, \qquad Q_{\mathrm{col}}(512) \approx 2^{170.7}, \qquad Q_{\mathrm{pre}}(512) \approx 2^{256} \tag{2.9}$$

$$digest = \sum_{j=0}^{15} \ell_j \, 2^{32j}, \qquad 0 \le \ell_j < 2^{32} \tag{2.10}$$

Legend:

- n — the digest length in bits; $Q_{\mathrm{pre}}, Q_{\mathrm{col}}$ — the idealized quantum query costs of preimage and collision search;
- ℓ_j — the 32-bit limbs of a 512-bit digest in the AIR; a single element of $\mathbb{F}_p$ would bind only about 61 bits.

2.5. Post-Quantum Soundness of the Proof Layer

Circle STARKs [14] use the circle group over $\mathbb{F}_p$, whose order for $p = 2^{61} - 1$ is 2^{61} (2.11). Soundness rests on hash binding and on algebraic tests; Shor's algorithm [50] applies to neither. The error decomposes additively over the binding, the Fiat–Shamir and FRI analysis in the QROM [51], and the algebraic tests (2.12) (A4).

$$C(\mathbb{F}_p) = \big\{(u, v) \in \mathbb{F}_p^2 : u^2 + v^2 = 1\big\}, \qquad |C(\mathbb{F}_p)| = p + 1 = 2^{61} \tag{2.11}$$

$$\varepsilon_{\mathrm{STARK}} \le \varepsilon_{\mathrm{bind}} + \varepsilon_{\mathrm{QROM}} + \varepsilon_{\mathrm{alg}} \tag{2.12}$$

Legend:

- $C(\mathbb{F}_p)$ — the circle group; $|C(\mathbb{F}_p)| = p + 1$ because $p \equiv 3 \pmod 4$;
- $\varepsilon_{\mathrm{bind}}$ — the probability of breaking the quantum binding of commitments and Merkle trees;
- $\varepsilon_{\mathrm{QROM}}$ — the soundness error of Fiat–Shamir and FRI in the QROM, including reduction losses and admitted attempts;
- $\varepsilon_{\mathrm{alg}}$ — the error of the algebraic tests over $\mathbb{F}_{p^s}$ (3.12).

3. Module 1 — Compute Primitive and Circle STARK

3.1. Canonical Tile

The native work unit is an exact integer product, (3.1). Its range excludes modular aliasing in $\mathbb{F}_p$, (3.2)–(3.3). Under the classical algorithm it costs n^3 multiply–accumulate operations, (3.4). The value -128 is excluded, because $512 \cdot 128^2 = 2^{23}$ would violate the strict bound.

$$X, Y \in \{-127, \ldots, 127\}^{512 \times 512}, \qquad Z = XY \in \mathbb{Z}^{512 \times 512} \tag{3.1}$$

$$|Z_{ij}| \le 512 \cdot 127^2 = 8\,258\,048 < 2^{23} \tag{3.2}$$

$$|Z_{ij} - Z'_{ij}| < 2^{24} < p \;\Rightarrow\; \big(Z_{ij} \equiv Z'_{ij} \pmod p \iff Z_{ij} = Z'_{ij}\big) \tag{3.3}$$

$$C_{\mathrm{mul}} = n^3 = 512^3 = 2^{27} \tag{3.4}$$

Legend:

- X, Y — the INT8 input matrices; Z — the canonical INT32 product; Z_{ij} — an entry;
- Z' — any admissible result, that is, any matrix satisfying (3.2);
- $p = 2^{61} - 1$ — the base modulus; $n = 512$ — the tile dimension;
- C_{mul} — the classical multiply–accumulate count of one tile.

3.2. Stochastic Verification

Six independent rounds over $\mathbb{F}_p$ bound the acceptance of a fixed false product by p^{-6} (3.5)–(3.8) [10], [11]. The vectors are derived after X, Y and Z are bound, from a canonical Fiat–Shamir transcript [12] with one domain per round and rejection sampling. The choice $r_F = 6$ raises the quantum search cost to 2^{183} (3.9).

$$v^{(\ell)} \xleftarrow{\mathrm{R}} \mathbb{F}_p^n, \qquad X\big(Yv^{(\ell)}\big) = Zv^{(\ell)}, \qquad \ell = 1, \ldots, r_F = 6 \tag{3.5}$$

$$D = XY - Z \neq 0 \;\Rightarrow\; \Pr[Dv = 0] = p^{-\operatorname{rank}(D)} \le p^{-1} \tag{3.6}$$

$$\Pr[\text{accept} \mid Z \neq XY] \leq p^{-6} \approx 2^{-366}, \qquad \Pr[\text{accept within } Q \text{ attempts}] \leq Q\,p^{-6} \tag{3.7}$$

$$C_{\text{Frei}} = 3\,r_F\,n^2 = 4\,718\,592 = \tfrac{9}{256}\,C_{\text{mul}} \tag{3.8}$$

$$Q_{\text{search}}^{\text{Frei}} \sim p^{r_F/2} = p^3 \approx 2^{183} \tag{3.9}$$

Legend:

- $v^{(\ell)}$ — the challenge vector of round ℓ; $\xleftarrow{\text{R}}$ — uniform sampling after binding;
- D — the error matrix; $\text{rank}(D)$ — its rank over $\mathbb{F}_p$;
- Q — the number of false transcripts tested classically;
- C_{Frei} — the field multiplications of the six rounds;
- $Q_{\text{search}}^{\text{Frei}}$ — the quantum search cost of an accepting false transcript.

3.3. Mixed-Field Circle STARK

The trace lives in $\mathbb{F}_p$, and every verifier and FRI challenge lives in $\mathbb{F}_{p^8}$ (3.10) [13]–[15]. The defining polynomial is irreducible: the only prime divisor of 8 is 2, so Rabin's test [52] reduces irreducibility to (3.11), and both conditions hold in exact arithmetic. A degree-8 extension lifts the per-check quantum search cost above the 128-bit target, which degree 4 does not reach (3.12)–(3.13). The FRI code rate, folding schedule, queries, openings and masking are fixed by the versioned profile so that A4 holds [16]–[18], [53].

$$\mathbb{F}_{p^8} = \mathbb{F}_p[X]/(f), \qquad f(X) = X^8 + X + 9, \qquad \text{basis } 1, X, \ldots, X^7 \tag{3.10}$$

$$X^{p^8} \equiv X \pmod{f}, \qquad \gcd\big(f,\ X^{p^4} - X\big) = 1 \tag{3.11}$$

$$\varepsilon_{\text{alg}} \lesssim \frac{d}{p^m}, \qquad Q_{\text{search}} \sim \left(\frac{p^m}{d}\right)^{1/2} \tag{3.12}$$

$$\log_2 Q_{\text{search}} \approx 244 - \tfrac{1}{2}\log_2 d \quad (m = 8), \qquad \log_2 Q_{\text{search}} \approx 122 - \tfrac{1}{2}\log_2 d \quad (m = 4) \tag{3.13}$$

Legend:

- X — here the indeterminate of $\mathbb{F}_p[X]$; f — the defining polynomial;
- (3.11) — Rabin's conditions: $f \mid X^{p^8} - X$, and f has no irreducible factor of degree 1, 2 or 4;
- d — the degree of one algebraic check; m — the extension degree;
- ε_{alg} — the error of that check; Q_{search} — the quantum search cost of a bad challenge.

3.4. Error Budget and Tile Decomposition

The probability of accepting an incorrect result is bounded by the sum of the component errors, never by their product (3.14). Data availability has the separate budget ε_{DA}. Products larger than one tile decompose canonically into tile products and certified reductions (3.15)–(3.16). A 1024×1024 product therefore takes eight tile products and four reductions.

$$\varepsilon_{\text{compute}} \leq \varepsilon_{\text{STARK}} + \varepsilon_{\text{Freivalds}} + \varepsilon_{\text{commit}} \tag{3.14}$$

$$Z_{ij}^{(t)} = X_{it}\,Y_{tj}, \qquad C_{ij} = \sum_t Z_{ij}^{(t)} \tag{3.15}$$

$$N_{\text{mul}} = \left\lceil \frac{m}{512} \right\rceil \left\lceil \frac{k}{512} \right\rceil \left\lceil \frac{n_o}{512} \right\rceil \tag{3.16}$$

Legend:

- $\varepsilon_{\text{compute}}$ — the acceptance probability of an incorrect result;
- $\varepsilon_{\text{STARK}}$ (2.12), $\varepsilon_{\text{Freivalds}}$ (3.7), $\varepsilon_{\text{commit}}$ — the component errors;
- X_{it}, Y_{tj} — 512×512 blocks; $Z_{ij}^{(t)}$ — a partial product; C_{ij} — an output block;
- $m \times k$, $k \times n_o$ — the operand dimensions; N_{mul} — the number of tile products.

3.5. Prover Memory

The conforming native-tile prover peaks at 19.5 GiB, which fits within consumer GPU memory (3.17)–(3.19). The trace width covers every auxiliary, range and lookup column. Extension-field values are accounted in M_{rest}. A trace of 2^{21} rows would double M_{LDE} to 32 GiB, so the primitive is the 512×512 tile.

$$M_{\mathrm{tr}} = N_{\mathrm{tr}}\, W_{\mathrm{tr}} \cdot 8 \text{ B} = 2^{20} \cdot 2^{7} \cdot 2^{3} \text{ B} = 1 \text{ GiB} \tag{3.17}$$

$$M_{\mathrm{LDE}} = 2\, B_{\mathrm{LDE}}\, N_{\mathrm{tr}}\, W_{\mathrm{tr}} \cdot 8 \text{ B} = 2 \cdot 2^{33} \text{ B} = 16 \text{ GiB} \tag{3.18}$$

$$M_{\mathrm{peak}} \le M_{\mathrm{tr}} + M_{\mathrm{LDE}} + M_{\mathrm{rest}} \le (1 + 16 + 2.5) \text{ GiB} = 19.5 \text{ GiB} \tag{3.19}$$

Legend:

- $N_{\mathrm{tr}} = 2^{20}$ — trace rows; $W_{\mathrm{tr}} = 128$ — trace columns; 8 bytes per base element;
- $B_{\mathrm{LDE}} = 8$ — the STARK blow-up factor, distinct from the P-DAL rate;
- $M_{\mathrm{tr}}, M_{\mathrm{LDE}}, M_{\mathrm{rest}}, M_{\mathrm{peak}}$ — the memory of the trace, of the two LDE buffers, of the remaining buffers (at most 2.5 GiB) and the peak.

3.6. Scientific Virtual Machine (SciVM) and the Ozaki-II Profile

The Scientific Virtual Machine (SciVM) deterministically executes admitted commercial and scientific tasks, carrying zero consensus weight (1.1) and remaining subject to Module 5 (Section 7). Each task binds a complete numerical execution profile Π_{num}, making floating-point reproducibility a property of specified execution semantics rather than a particular hardware platform or library. Supported dense high-precision matrix operations use a versioned Ozaki-II profile [1] over the native 512×512 INT8-input, INT32-accumulation tiles (3.1), without changing $\mathbb{F}_p$, $p = 2^{61} - 1$, or Module 3 (Section 5). The proof certifies the complete computation, including input conversion, tile products, CRT reconstruction, canonical rounding, and canonical semantics for all nonlinear, reduction, branching, optimizer, and PRNG steps.

$$\Pi_{\mathrm{num}} = \big(\nu, \mathsf{Enc}, \mathcal{D}, \mathsf{Scale}, \mathbf{q}, \mathcal{T}, \mathsf{Acc}, \mathcal{B}, \mathsf{CRT}_0, \mathsf{Rnd}, \mathsf{Exc}, \mathsf{Sem}, \mathsf{PRNG}, \mathcal{E}\big), \qquad \mathbf{q} = (q_1, \dots, q_r) \tag{3.20}$$

$$\begin{aligned}
&(\widetilde{X}, \widetilde{Y}, \alpha) = \mathsf{Scale}_{\Pi_{\mathrm{num}}}(X, Y), \qquad (A_{j,\ell}, B_{j,\ell}) = \mathsf{Tile}_{\mathcal{T}}\big(\mathrm{cres}_{q_j}(\widetilde{X}), \mathrm{cres}_{q_j}(\widetilde{Y})\big)_{\ell} \\
&T_{j,\ell} = \mathrm{lift}_p\big(A_{j,\ell} B_{j,\ell} \bmod p\big), \qquad R_j = \mathsf{Acc}_{\Pi_{\mathrm{num}}}\big((T_{j,\ell})_{\ell}\big) \bmod q_j \\
&Q = \prod_{j=1}^{r} q_j, \qquad Z_0 = \mathsf{CRT}_{0,Q}(R_1, \dots, R_r), \qquad \widehat{Z} = \mathsf{Unscale}_{\alpha}(Z_0), \qquad Z = \mathsf{Rnd}_{\Pi_{\mathrm{num}}}(\widehat{Z})
\end{aligned} \tag{3.21}$$

$$\begin{aligned}
&r \ge 1, \qquad 2 \le q_j \le 255, \qquad \gcd(q_i, q_j) = 1 \quad (i \ne j) \\
&\mathrm{cres}_q(z) = z - q \left\lfloor \frac{z + \lfloor q/2 \rfloor}{q} \right\rfloor, \qquad T_{j,\ell} = \mathrm{lift}_p\big(A_{j,\ell} B_{j,\ell} \bmod p\big) = A_{j,\ell} B_{j,\ell} \quad \text{over } \mathbb{Z} \\
&A_{j,\ell}, B_{j,\ell} \in \{-127, \dots, 127\}^{512 \times 512}, \qquad |T_{j,\ell}|_{\max} \le 512 \cdot 127^2 = 8\,258\,048 < 2^{23} < 2^{31} < p/2 \\
&|\widetilde{X}|_{\max} \le B_X, \qquad |\widetilde{Y}|_{\max} \le B_Y, \qquad B_{\mathrm{rec}} \ge K B_X B_Y \\
&Q = \prod_{j=1}^{r} q_j, \qquad 2B_{\mathrm{rec}} < Q \le \mathrm{lcm}(1, \dots, 255), \qquad |\widehat{Z}_{ab} - (XY)_{ab}| \le \varepsilon_{ab} \\
&\mathsf{CR}_{\Pi_{\mathrm{num}}} = 1 \;\Rightarrow\; \mathsf{Rnd}_{\Pi_{\mathrm{num}}}(\widehat{Z}_{ab} - \varepsilon_{ab}) = \mathsf{Rnd}_{\Pi_{\mathrm{num}}}(\widehat{Z}_{ab} + \varepsilon_{ab}) = Z_{ab} \quad \text{for every } a, b
\end{aligned} \tag{3.22}$$

$$\begin{aligned}
\tau &= (P, \mathsf{Enc}(x), \Pi_{\mathrm{num}}, s_{\mathrm{in}}, s_{\mathrm{out}}, y), \qquad \Delta\mathcal{W}_{\mathrm{consensus}}(\tau) = 0 \\
\mathcal{R}_{\mathrm{SciVM}}(\tau, w) = 1 &\iff \big[(x, s_{\mathrm{in}}) \in \mathcal{D}\big] \wedge \big[\mathsf{TraceCheck}_{\Pi_{\mathrm{num}}}(P, x, s_{\mathrm{in}}, s_{\mathrm{out}}, y; w) = 1\big] \\
\mathsf{Accept}_{\mathrm{SciVM}}(\tau, \pi) = 1 &\iff \big[\mathsf{Admit}_{\mathrm{Mod5}}(\tau) = 1\big] \wedge \big[\mathsf{Verify}_{\mathcal{R}_{\mathrm{SciVM}}}(\tau, \pi) = 1\big]
\end{aligned} \tag{3.23}$$

Legend:

- Π_{num} — the task-bound numerical profile; ν fixes its version, including the Ozaki-II construction; Enc fixes canonical operand encoding; $\mathcal{D}$ specifies admissible inputs and initial states; Scale and $\mathsf{Unscale}_{\alpha}$ specify integer conversion and exact rescaling, with scaling metadata α.

- $\mathbf{q}$ — the r pairwise coprime integer auxiliary moduli ≤ 255; $\mathcal{T}$ fixes tile placement and zero-padding; Acc fixes assembly and accumulation; $\mathcal{B}$ specifies range, overflow, and reconstruction bounds. Auxiliary moduli do not replace p; quantities exceeding native ranges use range-checked limbs and carries.
- CRT_0 — entrywise centered CRT reconstruction; cres_q returns the canonical centered residue; lift_p returns the centered integer representative of a native field element. j indexes residue moduli and ℓ indexes tile pairs; R_j is the assembled product modulo q_j.
- X, Y — finite matrix operands interpreted as exact values of their declared encodings; K is their common inner dimension. $\widetilde{X}, \widetilde{Y}$ are scaled integer operands; $B_X, B_Y, B_{\mathrm{rec}}$ are certified non-negative bounds. Accepted exact CRT reconstructions require $Q > 2B_{\mathrm{rec}}$; this reconstructs the scaled integer product and does not by itself eliminate input-conversion error.
- Rnd — the target format, monotone rounding rule, tie handling, and canonical output encoding; Exc fixes exceptional-value, subnormal, overflow, and underflow behavior. The ordinary finite branch obeys (3.21)–(3.22); exceptional branches must follow their committed semantics or reject.
- $\mathcal{E}$ — the declared accuracy contract and certified error bounds $\varepsilon_{ab} \geq 0$; bounds must be derived and checked inside $\mathsf{TraceCheck}$ from the committed conversion and execution, never trusted as unverified metadata. $\mathsf{CR}_{\Pi_{\mathrm{num}}}$ identifies a correctly rounded output claim. Its endpoint equalities are sufficient rounding-cell certificates. Exact unrounded claims require $\varepsilon_{ab} = 0$ and $Z_{ab} = \widehat{Z}_{ab}$.
- Sem — canonical semantics for all remaining instructions, including evaluation order, fused operations, nonlinear functions, reductions, control flow, and optimizer updates; PRNG fixes the generator, seed/state binding, and draw order. No implementation-dependent numerical choice is permitted.
- τ — the canonical task statement; profile Π_{num} is immutably bound by the client at admission. w is the complete execution witness and π its STARK proof. $\mathsf{TraceCheck}$ constrains every executed step; tile-only proofs do not satisfy the full-program relation. Insufficient reconstruction capacity $Q \leq 2B_{\mathrm{rec}}$ causes rejection.
- $\mathsf{Admit}_{\mathrm{Mod5}}$ — the existing admission and compensation conditions of Module 5 (Section 7); Verify uses the prescribed proof system and its soundness assumptions. Reproducibility means identical canonical outputs for identical bound programs, inputs, states, and profiles; it does not imply agreement with an unspecified external floating-point evaluation order.

4. Module 2 — Prover Data Availability Layer (P-DAL)

4.1. Encoding and Commitment

The payload is encoded with a rate-1/8 Reed–Solomon code [19] and committed by a SHA3-512 Merkle tree [20] with distinct leaf and node labels (4.1)–(4.2). A Merkle path proves membership of a fragment, and a proximity proof proves the encoding [15], [21].

$$\mathrm{RS}[n_{\mathrm{RS}},\ k_{\mathrm{RS}}], \qquad \frac{k_{\mathrm{RS}}}{n_{\mathrm{RS}}} = \frac{1}{8}, \qquad k_{\mathrm{RS}} = d + 1 \tag{4.1}$$

$$\mathit{root} = \mathsf{MerkleRoot}_{H_{512}}(c_1, \ldots, c_{n_{\mathrm{RS}}}), \qquad |A| \geq k_{\mathrm{RS}} \ \Rightarrow\ \mathsf{Decode}(\{c_i\}_{i \in A}) = \mathit{data} \tag{4.2}$$

Legend:

- $n_{\mathrm{RS}}, k_{\mathrm{RS}}$ — the code length and dimension; d — the maximal polynomial degree;
- c_i — the codeword symbols; *root* — the commitment bound in the manifest;
- A — a set of distinct authenticated symbol indices; Decode — Reed–Solomon reconstruction.

4.2. Data Availability Sampling

Light clients accept availability on the fast path by sampling authenticated fragments after the commitment is fixed [22], [23]. Under A6 the miss probability decays as 2^{-3s} (4.3)–(4.4). For example, $s = 43$ samples give $P_{\mathrm{miss}} < 2^{-129}$.

$$P_{\mathrm{miss}} = \binom{n_{\mathrm{RS}} - w}{s} \Big/ \binom{n_{\mathrm{RS}}}{s} \leq \left(1 - \frac{w}{n_{\mathrm{RS}}}\right)^s \tag{4.3}$$

$$n_{\mathrm{RS}} - w < k_{\mathrm{RS}} = \frac{n_{\mathrm{RS}}}{8} \ \Rightarrow\ P_{\mathrm{miss}} < 8^{-s} = 2^{-3s} \tag{4.4}$$

Legend:

- P_{miss} — the probability that every sample hits an available symbol although w symbols are withheld;

- w — the number of withheld symbols; s — the number of samples drawn without replacement;
- $n_{\mathrm{RS}} - w < k_{\mathrm{RS}}$ — the condition for unrecoverable data.

4.3. Reconstruction Rule

Consensus nodes admit a block to the fork choice only after physical reconstruction and full verification (4.5). Sampling success replaces no conjunct.

$$\begin{aligned}\mathsf{Admit}(\mathsf{blk}) \iff |A| \geq k_{\mathrm{RS}} \ \wedge\ \mathsf{Commit}\big(\mathsf{Decode}(A)\big) = \mathit{root} \\ \wedge\ \mathsf{VerifyProofs}(\mathsf{blk}) \ \wedge\ \mathsf{VerifyTransitions}(\mathsf{blk})\end{aligned} \tag{4.5}$$

Legend:

- blk — a candidate block; A — the retrieved authenticated fragments;
- Commit — recomputation of the root from the decoded data and their encoding;
- $\mathsf{VerifyProofs}, \mathsf{VerifyTransitions}$ — verification of every proof and every data-dependent state transition.

4.4. Fast-Path Certificate F1

A result certificate separates a branch-invariant correctness relation from a branch-dependent payment relation (4.6)–(4.7). Delivery latency is bounded below by the remaining proving, transport and verification (4.8). Balance finality follows the confirmation depth of Section 5.6.

$$\mathrm{ComputeValid}_{\mathcal{C}}(\kappa) = \mathrm{ComputeValid}_{\mathcal{C}'}(\kappa) \quad \forall \mathcal{C}, \mathcal{C}', \qquad \kappa = (\mathit{program},\ \mathit{inputRoots},\ \mathit{outputRoot},\ \pi) \tag{4.6}$$

$$\mathsf{Pay} > 0 \ \Rightarrow\ \mathrm{PaymentValid}(\Sigma_{\mathcal{C}^*},\ \mathit{jobID},\ \mathit{tileID},\ \mathit{leaseVersion},\ \mathit{executor},\ \mathit{receiptID}) = 1 \tag{4.7}$$

$$t_{\mathrm{ready}} \gtrsim t_{\mathrm{proof,rem}} + t_{\mathrm{transport}} + t_{\mathrm{verify}}, \qquad t_{\mathrm{transport}} \geq \frac{|\text{remaining data}|}{\text{bandwidth}} \tag{4.8}$$

Legend:

- $\mathcal{C}, \mathcal{C}'$ — any two branches; $\mathcal{C}^*$ — the canonical branch (5.9); $\Sigma_{\mathcal{C}^*}$ — its state; Pay — a payout;
- κ — the certificate: the program, the committed inputs and output, and the proof π;
- $\mathit{jobID}, \mathit{tileID}, \mathit{leaseVersion}, \mathit{executor}, \mathit{receiptID}$ — the identifiers of the economic claim;
- t_{ready} — the time to a verified result at the client; $t_{\mathrm{proof,rem}}, t_{\mathrm{transport}}, t_{\mathrm{verify}}$ — its components.

5. Module 3 — Dual-Lottery Consensus and Anti-Grinding Fresh Work

5.1. Fresh Work Context and Three Diagnostic Generators

The Internal Oracle is a public, versioned generator anchored in chain state; no external oracle exists. Let $k = h - h_{\mathrm{G}} \geq 1$, $j_h = (k-1) \bmod 3 \in \{0,1,2\}$, and $g_h = 3100 + j_h$. Before any matrix computation, every attempt binds sixteen fields into x, among them *attemptNonce*, *bodyCoreRoot* and the active generator profile g_h (5.1). The seed is derived via domain-separated SHA3-512 and expanded into independent SHAKE256 streams $\mathcal{R}_A, \mathcal{R}_B$ (5.2).

$$\begin{aligned}x = \mathsf{Enc}\big(&\mathit{chainID},\ \mathit{genesisID},\ \mathit{protocolVersion},\ \mathit{cryptoSuiteID},\\ &\mathit{programID},\ \mathit{AIR_ID},\ g_h,\\ &\mathit{parentCanonicalID},\ \mathit{stableAnchorID},\ h,\\ &\mathit{minerPolicyDigest},\ \mathit{payoutPolicyDigest},\ \mathit{attemptNonce},\\ &\mathit{bodyCoreRoot},\ \mathit{parentDerivedDifficultyContext},\ \mathit{timeBucket}\big)\end{aligned} \tag{5.1}$$

$$\mathit{seed} = H_{512}\big(\mathsf{Enc}(\texttt{FW/INPUT},\ x)\big), \qquad \mathcal{R}_A = \mathsf{SHAKE256}\big(\mathsf{Enc}(\texttt{FW/A},\ \mathit{seed})\big), \qquad \mathcal{R}_B = \mathsf{SHAKE256}\big(\mathsf{Enc}(\texttt{FW/B},\ \mathit{seed})\big) \tag{5.2}$$

Legend:

- $\mathit{chainID}, \mathit{genesisID}, \mathit{protocolVersion}, \mathit{cryptoSuiteID}$ — the chain, Genesis, protocol and suite domain;

- *programID*, *AIR_ID*, g_h — the program, the AIR and the modulo-3 diagnostic generator ID ($3100 + j_h$);
- *parentCanonicalID*, *stableAnchorID*, h — the parent, the stable anchor and the block height;
- *minerPolicyDigest*, *payoutPolicyDigest* — the miner and payout policies, which fix the beneficiary before work;
- *attemptNonce* — the nonce; *bodyCoreRoot* — the ordered operation body root fixed before the attempt;
- *parentDerivedDifficultyContext*, *timeBucket* — the difficulty context and the time bucket.

5.2. Instance Generation and Identifiers

The schedule $j_h = (h - h_{\mathrm{G}} - 1) \bmod 3$ rotates across three deterministic matrix profiles in $O(n^2)$ generation work (5.3a)–(5.3c):

- Profile 1 ($j_h = 0$, Signed outlier and small-value arithmetic): Each row of A and B is initialized with small background entries drawn from $\{-2, -1, 0, 1, 2\}$ according to (5.3a), after which a partial Fisher–Yates shuffle using unbiased rejection sampling ($\mathsf{UniformBelow}(512 - t)$ with at most 8 draws per step) selects 16 distinct column indices per row, overwriting the first eight with -127 and the remaining eight with $+127$ (3.125% row outlier density; 72.65625% expected overall zero fraction) [3].
- Profile 2 ($j_h = 1$, Nonnegative arithmetic graphs): Entries are drawn uniformly from $\{0, \ldots, 127\}$ by masking stream bytes (5.3b), forming weighted two-step path-count and transition-numerator kernels [2].
- Profile 3 ($j_h = 2$, Correlated dense accumulator stress): A carrier stream $\mathsf{SHAKE256}\big(\mathsf{Enc}(\texttt{PLAU/FW/CORRELATION/CARRIER},\ \mathit{seed})\big)$ fixes shared inner-dimension signs $\sigma_k \in \{-1, +1\}$, while operand streams draw independent uniform 3-bit magnitude deficits $E_{ik}, F_{kj} \in \{0, \ldots, 7\}$ (5.3c). Every accumulation term is positive, driving the expected entry $\mathbb{E}[C_{ij}] = 512(127 - 3.5)^2 = 7{,}809{,}152$ near the admissible kernel maximum 8,258,048 and defeating narrow 16-bit signed partial accumulators after three terms ($3 \cdot 120^2 = 43{,}200 > 32{,}767$) [3], [4].

$$\Pr[V = 0] = \frac{3}{4}, \qquad \Pr[V = -2] = \Pr[V = -1] = \Pr[V = 1] = \Pr[V = 2] = \frac{1}{16} \tag{5.3a}$$

$$A_{ik} = \mathcal{R}_A.\mathrm{byte}() \,\&\, 127, \qquad B_{kj} = \mathcal{R}_B.\mathrm{byte}() \,\&\, 127 \tag{5.3b}$$

$$\begin{gathered} E_{ik} = \mathcal{R}_A.\mathrm{byte}() \,\&\, 7, \qquad F_{kj} = \mathcal{R}_B.\mathrm{byte}() \,\&\, 7, \qquad A_{ik} = \sigma_k(127 - E_{ik}), \qquad B_{kj} = \sigma_k(127 - F_{kj}) \\ C_{ij} = \sum_{k=0}^{511} (127 - E_{ik})(127 - F_{kj}) \in [7{,}372{,}800,\ 8{,}258{,}048] \end{gathered} \tag{5.3c}$$

$$\begin{gathered} C = AB \in \mathbb{Z}^{512\times 512}, \qquad |C_{ij}| \le 8{,}258{,}048 \\ \mathit{outputRoot} = H_{512}\big(\mathsf{Enc}(\texttt{FW/OUTPUT},\ C)\big), \qquad \mathit{WorkID} = H_{512}\big(\mathsf{Enc}(\texttt{FW/ID},\ x,\ \mathit{outputRoot})\big) \\ \mathit{BlockID} = H_{512}\big(\mathsf{Enc}(\texttt{BLOCK},\ x,\ \mathit{outputRoot})\big), \qquad \mathit{ShareID} = \mathit{WorkID} \end{gathered} \tag{5.3}$$

Legend:

- $A, B \in \{-127, \ldots, 127\}^{512\times 512}$ — the generated operand matrices for profile j_h; C — the canonical INT32 product;
- $\sigma_k \in \{-1, +1\}$ — the inner-dimension carrier sign for Profile 3; $E_{ik}, F_{kj} \in \{0, \ldots, 7\}$ — independent uniform 3-bit magnitude deficits (admitting 9-pair bitplane popcount evaluation in $O(n^3/w + n^2)$ word operations, absorbed symmetrically by A1);
- *outputRoot* — the full-output commitment to C; *WorkID* = *ShareID* — the work identifier; *BlockID* — the block identifier.

5.3. Dual-Lottery and Six-Round Freivalds Verification

One verified evaluation enters two domain-separated lotteries in the style of FruitChains [5] (5.4)–(5.5). Consensus validators receive the complete 1,048,576-byte matrix C, check entry bounds $|C_{ij}| \le 8{,}258{,}048$, verify the lottery ticket against the parent-derived target, regenerate (A, B) in $O(n^2)$ work, and execute six Fiat–Shamir Freivalds rounds over $\mathbb{F}_p$ (3.5) without recomputing AB. In the nominal profile the targets stand in the ratio 120 (5.6).

$$\vartheta_S = H_{512}\big(\mathsf{Enc}(\texttt{FW/SHARE},\ \mathit{WorkID})\big), \qquad \vartheta_B = H_{512}\big(\mathsf{Enc}(\texttt{FW/SEAL},\ \mathit{WorkID})\big) \tag{5.4}$$

$$\begin{gathered} \mathsf{VerifyFW}(x, C) \iff \mathsf{Available}(C) \wedge \mathsf{RangeOK}(C) \wedge \mathsf{Freivalds}_6(A, B, C) = 1 \\ \mathsf{ShareValid} \iff \mathsf{VerifyFW}(x, C) \wedge \vartheta_S < \Theta_S, \qquad \mathsf{SealValid} \iff \mathsf{VerifyFW}(x, C) \wedge \vartheta_B < \Theta_B \end{gathered} \tag{5.5}$$

$$q = \frac{\Theta}{2^{512}}, \qquad \lambda_B = \Lambda q_B, \quad \lambda_S = \Lambda q_S, \qquad T_B = \lambda_B^{-1} = 120 \text{ s}, \quad \lambda_S = 1 \text{ s}^{-1}, \quad \frac{\Theta_S}{\Theta_B} = 120 \tag{5.6}$$

Legend:

- ϑ_S, ϑ_B — the share and seal lottery values in $[0, 2^{512})$; $\Theta_S, \Theta_B = 2^{384}\theta_{S,B}$ — the parent-derived targets;
- $\mathsf{VerifyFW}(x, C)$ — $O(n^2)$ consensus verification combining availability, INT32 range checks and six Freivalds rounds;
- q_S, q_B — the per-attempt success probabilities; Λ — the effective attempt rate;
- λ_B, λ_S — the seal and share rates; T_B — the mean block interval.

5.4. Witness-Independent Lottery Identifiers

Every lottery value and identifier is a deterministic function of x, and free witness material enters none of them (5.7)–(5.8). Post-computation witness grinding is therefore impossible by Theorem 5.1, while pre-computation context grinding over x is bound by cost assumption A1.

Theorem 5.1 (Witness-independent lottery identifiers). For every context x and any two valid witnesses w, w' of the same statement, (5.7) holds. Each distinct evaluated context therefore yields exactly one draw per lottery (5.8).

Proof. *seed* depends on x by (5.2), and (A, B) depends on *seed* by (5.3a)–(5.3c). *outputRoot*, *WorkID*, *BlockID* and $\vartheta_{B,S}$ depend on (x, C) with $C = AB$ by (5.3)–(5.4). The witness does not occur in these maps.

$$\Phi(x, w) = \Phi(x, w'), \qquad \Phi = \big(\mathit{WorkID},\ \mathit{BlockID},\ \vartheta_B,\ \vartheta_S\big) \tag{5.7}$$

$$\#\{\text{draws per lottery}\} = \#\{\text{distinct } x \text{ evaluated by } \mathsf{Eval}_{\mathrm{FW}}\} \tag{5.8}$$

Legend:

- w, w' — free witness material: signature bytes, prover randomness, Merkle-path order, proof serialization;
- Φ — the tuple of identifiers and lottery values;
- $\mathsf{Eval}_{\mathrm{FW}}$ — the canonical evaluation (5.2)–(5.3); the residual cost of an attempt is governed by A1.

5.5. Fork Choice

Only seals weigh history (5.9). Heaviest-chain selection follows [28]–[30]. Difficulty and windows follow the variable-power FruitChains rules [6], with the fixed target ratio of (5.6). Shares, commercial results, fees and bonds carry zero chain weight (5.10).

$$d_B(\mathsf{blk}) = \frac{2^{512}}{\Theta_B(\mathsf{blk})}, \qquad \mathcal{W}(\mathcal{C}) = \sum_{\mathsf{blk} \in \mathcal{C}} d_B(\mathsf{blk}), \qquad \mathcal{C}^* = \arg \max_{\mathcal{C} \text{ fully validated}} \mathcal{W}(\mathcal{C}) \tag{5.9}$$

$$d_S = \frac{2^{512}}{\Theta_S}, \qquad \omega(\text{seal}) = d_B, \qquad \omega(\text{share}) = \omega(\text{result}) = \omega(\text{fee}) = \omega(\text{bond}) = 0 \tag{5.10}$$

Legend:

- $d_B(\mathsf{blk})$ — the seal weight of block blk; $\mathcal{W}(\mathcal{C})$ — the cumulative chain weight;
- $\mathcal{C}^*$ — the selected branch; on a tie the current valid tip is kept;
- d_S — the share weight, used only for rewards and rights; ω — the contribution of an object to chain weight.

5.6. Delay and Reorganization Bound

The rare seal keeps the fork overlap small (5.11). At constant rates, Theorem 1 of [7] bounds reorganizations under A2–A3 (5.12)–(5.13), and the sufficient depths in the nominal profile follow from (5.14) [31]–[34].

$$P_{\text{near}} = 1 - e^{-\Delta_{\text{eff}}/T_B} \approx 1.653\% \ \ (\Delta_{\text{eff}} = 2 \text{ s}), \qquad T_B \geq 60\,\Delta_{\text{eff}} \tag{5.11}$$

$$u = (1 - \alpha)\, e^{-\Delta_{\text{eff}}/T_B} > \tfrac{1}{2} \iff \alpha < 1 - \tfrac{1}{2} e^{\Delta_{\text{eff}}/T_B} \approx 0.4916 \tag{5.12}$$

$$P_{\text{reorg}}(k) \le \min\left\{1,\ \left(2+2\sqrt{\frac{u}{1-u}}\right)\left[4u(1-u)\right]^{k}\right\} \tag{5.13}$$

$$k^{*} = \left\lceil \frac{\ln\!\big((2+2\sqrt{u/(1-u)})/\varepsilon_{\text{set}}\big)}{-\ln\!\big(4u(1-u)\big)} \right\rceil, \qquad k^{*}(0.10)=34, \quad k^{*}(0.25)=115, \quad k^{*}(0.40)=886 \tag{5.14}$$

Legend:

- Δ_{eff} — the bound on time from discovery to full validation; $T_B = 120$ s;
- P_{near} — the probability of overlapping discoveries; α — the adversarial share of effective seal capacity;
- u — the delay-discounted honest share; k — the confirmation depth; $P_{\text{reorg}}(k)$ — the reorganization probability;
- $k^{*}(\alpha)$ — the sufficient depth for $\varepsilon_{\text{set}} = 2^{-40}$ and $\Delta_{\text{eff}} = 2$ s, that is 68, 230 and 1,772 minutes.

5.7. Rewards and Deviation Bound

Rewards are proportional to mature integer share weight w_s, computed exactly without aggregation bias via the per-lot integer rules of Module 5 (Section 7.8). The block author receives no leader premium and captures no fees. On the fairness event, deviation gains at most $\delta_f E_e$ (5.16) [5], [6], [8], [35].

$$w_{i,e} = \sum_{s\in\mathcal{S}_e,\ \text{owner}(s)=i} w_s, \qquad W_e = \sum_{s\in\mathcal{S}_e} w_s \tag{5.15}$$

Theorem 5.2 (Deviation bound). On $\mathcal{F}_\delta$ (A5), a coalition of share α satisfies (5.16).

Proof. The honest remainder receives at least $(1-\delta_f)(1-\alpha)E_e$, which gives the first bound. Honest play yields at least $(1-\delta_f)\alpha E_e$. Subtracting the two gives the result.

$$P_A^{\text{dev}} \le \big[\alpha + \delta_f(1-\alpha)\big]E_e, \qquad P_A^{\text{hon}} \ge (1-\delta_f)\,\alpha\,E_e, \qquad P_A^{\text{dev}} - P_A^{\text{hon}} \le \delta_f E_e \tag{5.16}$$

Legend:

- $\mathcal{S}_e$ — the mature qualifying shares of window e; owner(s) — the beneficiary bound in x;
- w_s — the integer weight of share s (6.1); $w_{i,e}$ — the mature integer weight of participant i; W_e — the total mature integer weight of epoch e; E_e — the reward budget (7.21);
- δ_f — the fairness tolerance; $P_A^{\text{dev}}, P_A^{\text{hon}}$ — the coalition payout when deviating and when honest.

6. Module 4 — Deterministic Scheduler and D'Hondt Allocation

6.1. ShareID-Indexed Weights

The unit of execution entitlement is a discrete, immutable ShareID of verified mature work, never an address. Its integer weight is fixed by the consensus target of its parent (6.1). Owners pledge bonds for concurrent leases, and the batch size is bounded by budget, readiness and capacity (6.2).

$$w_s = \left\lfloor \frac{2^{512+128}}{\Theta_S(s)} \right\rfloor = \left\lfloor 2^{128}\, d_S(s) \right\rfloor, \qquad W = \sum_{s\in\mathcal{S}} w_s, \qquad \frac{2^{128} d_S(s) - w_s}{2^{128} d_S(s)} < 2^{-128} \tag{6.1}$$

$$B_i \ge K_i\, b_{\text{lease}}, \qquad N = \min\Big\{N_e^{\text{svc}},\ N_{\text{ready}},\ \sum_{i\in\mathcal{O}(\mathcal{S})} K_i^{\text{free}}\Big\}, \qquad N_e^{\text{svc}} \le \theta\,\frac{W_e}{c_{\text{ref}}} \tag{6.2}$$

Legend:

- s — a source, identified by its *ShareID*; $\mathcal{S}$ — the snapshot of mature qualifying sources; $\Theta_S(s)$ — its consensus target;
- w_s — the integer weight; W — the total weight; $d_S(s)$ — the share weight (5.10);
- B_i — the bond of owner i; K_i — its concurrent-lease capacity; b_{lease} — the bond per lease;
- N — the feasible slots of a batch; N_e^{svc} — the service budget; N_{ready} — the ready tiles;
- $\mathcal{O}(\mathcal{S})$ — the owners holding a source in $\mathcal{S}$; K_i^{free} — the remaining capacity;
- W_e — the mature share weight of epoch e; c_{ref} — the reference weight per service unit; θ — a deployment factor.

6.2. Floor-Quota Defect

Floor quotas lose up to $|\mathcal{S}| - 1$ slots and can assign none when mining power is fragmented (6.3). A per-wallet largest-remainder rule would assign all slots but would be manipulable by moving weight between addresses. Both rules are replaced by (6.4).

$$N - \sum_{s \in \mathcal{S}} \left\lfloor \frac{N w_s}{W} \right\rfloor < |\mathcal{S}|, \qquad w_1 = \cdots = w_9,\ N = 8 \ \Rightarrow\ \left\lfloor \frac{8 w_s}{W} \right\rfloor = 0 \ \ \forall s \tag{6.3}$$

Legend:

- $|\mathcal{S}|$ — the number of sources; the left-hand side of (6.3) is the number of slots a floor rule leaves unassigned;
- the example — nine equal sources and eight slots, where every floor quota is zero.

6.3. Sequential D'Hondt Recursion

Each slot is assigned to the eligible ShareID with the largest D'Hondt quotient $w_s/(a_s + 1)$, with ties broken by the smallest ShareID (6.4). Quotients are compared exactly (6.5). The first batch of a cohort admits the floor-plus-remainder shortcut (6.6). When owner limits bind, the recursion (6.4) is normative.

$$\begin{aligned} s_t^* &= \arg\max_{s \in \mathcal{E}_t} \frac{w_s}{a_s^{(t)} + 1} \quad \text{(ties: least } \mathit{ShareID}_s\text{)}, \qquad \mathcal{E}_t = \left\{ s \in \mathcal{S} : K^{(t)}_{\mathrm{owner}(s)} > 0 \right\} \\ a_s^{(t+1)} &= a_s^{(t)} + \mathbb{1}[s = s_t^*], \qquad K_i^{(t+1)} = K_i^{(t)} - \mathbb{1}[i = \mathrm{owner}(s_t^*)] \\ \mathsf{Lease}_t &= \big(\mathit{tile}_t,\ \mathit{leaseVersion},\ \mathrm{owner}(s_t^*)\big), \qquad t = 0, 1, \ldots \ \text{ while } \ t < N \ \wedge \ \mathcal{E}_t \neq \varnothing \end{aligned} \tag{6.4}$$

$$\frac{w_s}{a_s + 1} > \frac{w_r}{a_r + 1} \iff w_s\,(a_r + 1) > w_r\,(a_s + 1) \tag{6.5}$$

$$b_s = \left\lfloor \frac{N w_s}{W} \right\rfloor, \qquad N_{\mathrm{rem}} = N - \sum_{s \in \mathcal{S}} b_s < |\mathcal{S}|, \qquad a_s^{(0)} = b_s \text{ then } N_{\mathrm{rem}} \text{ steps of (6.4)} \tag{6.6}$$

Legend:

- t — the slot index; s_t^* — the source served at step t; $\mathcal{E}_t$ — the eligible sources;
- $a_s^{(t)}$ — the slots held by s, persisting across the batches of a cohort; $K_i^{(t)}$ — the remaining capacity of owner i;
- $\mathit{ShareID}_s$ — compared as a canonical 64-byte string, fixed by bound work and not selectable after computation;
- Lease_t — the unique lease created atomically at step t; tile_t — the t-th canonical ready tile;
- r — any other source; b_s — the floor quota; N_{rem} — the remaining slots.

6.4. Completeness and Proportionality

The recursion assigns every feasible slot (6.7). Without binding capacity it selects the N_c smallest D'Hondt markers [54], which gives two-sided bounds and convergence to exact proportionality (6.8)–(6.9).

Theorem 6.1 (Completeness). Each step of (6.4) creates exactly one lease and increments exactly one counter, and $\mathcal{E}_t \neq \varnothing$ for every $t < N$ by (6.2); a batch therefore allocates exactly its N feasible slots (6.7).

Theorem 6.2 (Proportionality). For a fixed cohort without binding capacity, after N_c slots, (6.8) and (6.9) hold.

Proof. The selected markers $\nu_{s,j} = j/w_s$ are the N_c smallest. At most N_c markers do not exceed N_c/W, which gives the lower bound. Let ν^* be the N_c-th selected marker. Then $a_r \geq \nu^* w_r - 1$ for every r, so $N_c \geq \nu^* W - |\mathcal{S}|$; combined with $a_s \leq \nu^* w_s$ this gives the upper bound. Dividing by N_c yields (6.9).

$$\sum_{s \in \mathcal{S}} \big(a_s^{(N)} - a_s^{(0)}\big) = N \leq \sum_{i \in \mathcal{O}(\mathcal{S})} K_i^{\mathrm{free}} \tag{6.7}$$

$$\left\lfloor \frac{N_c\, w_s}{W} \right\rfloor \leq a_s(N_c) \leq \frac{(N_c + |\mathcal{S}|)\, w_s}{W} \tag{6.8}$$

$$\lim_{N_c\to\infty}\frac{a_s(N_c)}{N_c}=\frac{w_s}{W} \tag{6.9}$$

Legend:

- N_c — the cumulative slots allocated in the cohort; $a_s(N_c)$ — those held by s;
- $\nu_{s,j}$ — the j-th marker of s; ν^* — the last selected marker;
- $a_s^{(0)}, a_s^{(N)}$ — the counter of s before and after the batch; (6.7) — no feasible slot is left unassigned.

6.5. Aggregation Invariance

The allocation is computed over discrete ShareIDs, and owner labels enter (6.4) only through capacity. Without binding capacity, an entity's allocation is the sum over its ShareIDs, invariant under partition into owners or addresses (6.10). D'Hondt's large-unit bias cannot be captured by aggregation: the largest allocatable unit is a single share, and its weight depends only on the consensus target of its parent, not on holdings (6.11). With binding capacity, merging identities yields no premium over the best capacity-matched split (6.10), (6.12), provided collateral is uniquely backed globally ($\sum_i K_i \le \lfloor \sum_i B_i/b_{\text{lease}} \rfloor$).

Theorem 6.3 (No aggregation premium). Let an entity E hold $\mathcal{S}_E \subseteq \mathcal{S}$ with $W > 0$.

1. Without binding capacity, (6.10) and (6.11) hold for any two partitions P, P' of $\mathcal{S}_E$ into owners, and $\Pi_{\text{agg}} = 0$.
2. With binding capacity, a merged owner of free capacity K_E^{free} in a batch is reproduced by a partition whose free capacities sum to K_E^{free}. Hence $\Pi_{\text{agg}} \le 0$, and (6.12) holds for simultaneous active leases and batch allocations.

Proof. (1) Without binding capacity, (6.4) selects the N_c smallest elements of the marker multiset $\{(j/w_s, \mathit{ShareID}_s)\}$, which carries no owner label. Summing (6.8) over $\mathcal{S}_E$ gives (6.11). (2) Give each part the capacity it receives in the merged run. By induction on t, the split run selects the same s_t^*, because a part becomes ineligible only after it has received its final merged allocation. (6.12) follows from $K_i^{(t)} \ge 0$ and (6.13).

$$a_E^P(N_c) = a_E^{P'}(N_c) = \sum_{s\in\mathcal{S}_E} a_s(N_c), \qquad \Pi_{\text{agg}} := a_E^{\text{merged}} - \max_{P:\ \sum_k K_k = K_E} a_E^P \le 0 \tag{6.10}$$

$$w_s = \omega_\Theta\big(\Theta_S(\text{parent}(s))\big), \qquad \sum_{s\in\mathcal{S}_E}\left\lfloor \frac{N_c w_s}{W}\right\rfloor \le a_E(N_c) \le \frac{(N_c+|\mathcal{S}|)\,W_E}{W} \quad (W>0) \tag{6.11}$$

$$a_E^{\text{batch}} \le K_E^{\text{free}} = \sum_{i\in E} K_i^{\text{free}}, \qquad \big|\text{active}(E)\big| \le K_E = \sum_{i\in E} K_i \le \left\lfloor \frac{\sum_{i\in E} B_i}{b_{\text{lease}}}\right\rfloor \tag{6.12}$$

Legend:

- E — an entity controlling one or more keys; $\mathcal{S}_E$ — its ShareIDs; $W_E = \sum_{s\in\mathcal{S}_E} w_s$;
- P, P' — partitions of $\mathcal{S}_E$ into owners or addresses; a_E^P — the slots of E under P; K_k — the capacity of part k;
- Π_{agg} — the aggregation premium of one merged owner over the best capacity-matched split; it is 0 without binding capacity;
- ω_Θ — the weight map of (6.1); parent(s) — the parent block under which s was found;
- a_E^{batch} — slots allocated to E in a single batch without intervening capacity release; active(E) — simultaneously active leases of E; K_E, K_E^{free} — total and free bonded capacity of E.

6.6. Leases and Phase Machine

Each admitted tile has exactly one active lease bound to one executor, and the bond reserved across an executor's leases never exceeds its bond (6.13). The job lifecycle is the transition relation (6.14), and deadlines separate client input availability from executor computation (6.15). Every transition commits atomically in canonical block order under the versioned read-and-predicate guard (6.15a).

$$\begin{aligned}&\mathsf{Lease} = (\mathit{jobID},\ \mathit{tileID},\ \mathit{leaseVersion},\ \mathit{executor},\ \mathit{escrowID},\ \mathit{price},\ B_{\text{res}},\ h_{\text{deadline}})\\ &\big|\text{active}(\mathit{tileID})\big| \le 1, \qquad \sum_{\ell\in\text{active}(i)} B_{\text{res},\ell} \le B_i\end{aligned} \tag{6.13}$$

$$
\begin{aligned}
\bot &\xrightarrow{\texttt{ESCROW_FUND_AND_QUEUE}} \text{Funded}, \\
\text{Funded} &\xrightarrow{\texttt{JOB_ADMIT_AND_LEASE}} \text{Admitted}, \qquad \text{Funded} \xrightarrow{\text{TTL} \vee \text{cancel}} \text{Refunded}, \\
\text{Admitted} &\xrightarrow{\texttt{INPUT_READY},\ h \le h_{\text{admit}} + D_{\text{avail}}} \text{Running}, \qquad \text{Admitted} \xrightarrow{h > h_{\text{admit}} + D_{\text{avail}}} \text{Retry}, \\
\text{Running} &\xrightarrow{\texttt{RESULT_SETTLE}} \text{Settled}, \qquad \text{Running} \xrightarrow{\texttt{LEASE_TIMEOUT},\ h > h_{\text{deadline}}} \text{Retry}, \\
\text{Retry} &\xrightarrow{\texttt{LEASE_RETRY},\ \beta_{\text{retry}} > 0} \text{Admitted}, \qquad \text{Retry} \xrightarrow{\texttt{ESCROW_REFUND},\ \beta_{\text{retry}} = 0} \text{Refunded}
\end{aligned}
\tag{6.14}
$$

$$
\begin{aligned}
& h_{\text{ready}} \le h_{\text{admit}} + D_{\text{avail}}, \qquad h_{\text{deadline}} = h_{\text{ready}} + \text{TTL}_{\text{comp}}, \\
& \text{penalty}_{\text{executor}}\big(\text{Admitted} \to \text{Retry at } h > h_{\text{admit}} + D_{\text{avail}}\big) = 0
\end{aligned}
\tag{6.15}
$$

$$
\begin{aligned}
\mathsf{CommitAllowed}(t, \Sigma^-) \iff & \left(\bigwedge_{k \in \mathcal{D}_{\text{read}}(t)} \widehat{\nu}_{t,k} = \nu_k(\Sigma^-) \right) \wedge \mathsf{Pre}(t, \Sigma^-) \wedge \mathsf{Inv}\big(\mathsf{Apply}(t, \Sigma^-)\big) \\
& \nu'_k = \nu_k + 1 \qquad (k \in \mathcal{D}_{\text{write}}(t)), \qquad \mathcal{D}_{\text{write}}(t) \subseteq \mathcal{D}_{\text{read}}(t) \\
\mathsf{ShortcutAssignments}(\Sigma) = \mathsf{SequentialDHondtAssignments}(\Sigma)
\end{aligned}
\tag{6.15a}
$$

Legend:

- Lease — the lease tuple; B_{res} — its bond reservation; active$(\cdot)$ — the active leases of a tile or of executor i;
- $\bot$ — the empty job; Funded, Admitted, Running, Retry — working states; Settled, Refunded — terminal states;
- h — the canonical height; h_{admit} — the height of `JOB_ADMIT_AND_LEASE`; D_{avail} — the data window;
- h_{ready} — the height of `INPUT_READY`; TTL_{comp} — the computation TTL; h_{deadline} — the delivery deadline; zero executor penalty in (6.15) applies solely to the input-unavailable transition Admitted → Retry;
- β_{retry} — the remaining retry budget authorized at funding; each retry is a paid re-admission with a new *leaseVersion*;
- $\mathcal{D}_{\text{read}}(t), \mathcal{D}_{\text{write}}(t)$ — complete protocol-derived read and write sets covering cohort counters, owner capacities, collateral reservations, accounts, leases, queue state, and queue/cohort index versions for range and predicate reads (preventing phantom insertions). Expected versions $\widehat{\nu}_{t,k}$ are transaction-bound and versions never wrap;
- Σ^- — the live canonical prefix state. Version comparison, state application, and version increments form one indivisible commit in canonical block order, rejecting stale reads without effects. $\mathsf{ShortcutAssignments}$ requires exact equality of the ordered tile-to-executor assignments.

6.7. Ordering and Throughput

For a fixed canonical state the allocation is unique, and payment is bound to the lessee, which eliminates the execution race and priority auctions [36] (6.16). Queue inclusion rests on A3. Throughput is bounded by the narrowest resource (6.17).

$$
\mathsf{Alloc}(\Sigma) \text{ is a function of } \Sigma, \qquad \mathsf{Pay}(e, \mathit{tileID}) > 0 \ \Rightarrow \ e = \mathsf{Lease}(\mathit{tileID}).\mathit{executor}
\tag{6.16}
$$

$$
T_{\text{compute}} \approx \left\lceil \frac{M_T}{N_G} \right\rceil \tau_C, \qquad \text{Throughput} \le \min\Big\{ \sum_i \mu_i,\ \frac{B_{\text{DAL}}}{s_C},\ \min_v \frac{B_v}{s_C},\ \min_v \frac{\text{Compute}_v}{v_C} \Big\}
\tag{6.17}
$$

Legend:

- Alloc — the scheduler output; $\mathsf{Pay}(e, \mathit{tileID})$ — the payment to party e for a tile;
- T_{compute} — the ideal job time; M_T — the equal tiles; N_G — the executors; τ_C — the time per tile;
- μ_i — the rate of executor i; B_{DAL}, B_v — the bandwidth of P-DAL and of node v; Compute_v — the validation capacity of v;
- s_C, v_C — the data size and the validation cost per unit of work.

7. Module 5 — Transparent State, Transactions and Tokenomics

7.1. State and Operation Set

The state is a transparent tuple over a single native asset (7.1), and the operation set is closed: there is no EVM, no user contract, no user-issued token and no financial-privacy primitive (7.2).

$$\Sigma = \big(\mathcal{U},\, U_{\text{initial}},\, \mathcal{E},\, \mathcal{B},\, \mathcal{L},\, \mathcal{Q},\, \mathcal{V},\, R_a,\, R_l,\, P_{\text{pending}},\, R_I,\, R_M,\, \mathit{Emission},\, \mathit{InitialValve},\, \mathit{InitialNonce}\big), \quad \mathsf{Assets} = \{\text{PLAU}\} \tag{7.1}$$

$$\begin{aligned}\mathsf{Ops} = \{\ &\texttt{TWL_TRANSFER},\ \texttt{ESCROW_FUND_AND_QUEUE},\ \texttt{JOB_ADMIT_AND_LEASE},\ \texttt{INPUT_READY},\\ &\texttt{RESULT_SETTLE},\ \texttt{LEASE_TIMEOUT},\ \texttt{LEASE_RETRY},\ \texttt{ESCROW_REFUND},\\ &\texttt{BOND_LOCK},\ \texttt{BOND_RELEASE},\ \texttt{BOND_SLASH},\ \texttt{FW_REWARD},\ \texttt{RESERVE_MATURE},\\ &\texttt{FEE_REWARD_ALLOCATE},\ \texttt{INITIAL_POOL_WITHDRAWAL},\\ &\texttt{L2_DEPOSIT},\ \texttt{L2_BATCH},\ \texttt{L2_WITHDRAW},\ \texttt{L2_EXIT}\ \}\end{aligned} \tag{7.2}$$

Legend:

- $\mathcal{U}$ — ordinary public UTXOs (summed as U); U_{initial} — the protected `InitialProtocolPool` balance;
- $\mathcal{E}, \mathcal{B}$ — escrow and bond accounts ($E_{\text{esc}}, B_{\text{bond}}$); $\mathcal{L}, \mathcal{Q}$ — leases with scheduler counters, and the job queue; $\mathcal{V}$ — transparent L2 vaults (V_{L2});
- $R_a, R_l, P_{\text{pending}}$ — the available fee reserve, the locked fee reserve and the segregated reward payables;
- R_I, R_M — the unissued initial protocol reserve (`UnissuedInitial`) and unissued miner reserve (`UnissuedMiners`);
- *Emission*, *InitialValve*, *InitialNonce* — integer release bookkeeping, the rolling 720-block outflow ring buffer and the withdrawal replay counter;
- Ops — the closed operation set; primary release is an automatic per-block transition, not a user-callable operation.

7.2. Public TWL Authorization and Conflicts

Every public spend is valid only if each presented key matches the locking policy of the consumed output read from live state (7.3). The authorization message commits to the complete effect digest, so no weak signature-hash mode exists (7.4)–(7.5). Conflicts are local, which admits parallel validation alongside serialized valve updates (7.6) [28].

$$\forall\, op \in \mathit{inputOutpoints}:\ \ op \in \mathcal{U}_{\text{live}} \ \wedge\ \mathsf{Match}\big(pk_{op},\ \mathit{lockingPolicy}(op)\big) = 1 \tag{7.3}$$

$$\begin{aligned}\mathit{TxCore} = \mathsf{Enc}\big(&\mathit{chainID},\ \mathit{genesisID},\ \mathit{protocolVersion},\ \mathit{operationType},\\ &\mathit{accountID},\ \mathit{expectedAccountVersion},\ \mathit{amount},\ \mathit{fee},\\ &\#\mathit{inputs},\ \mathit{orderedInputs},\ \#\mathit{outputs},\ \mathit{orderedPublicOutputs},\ \mathit{payload}\big)\end{aligned} \tag{7.4}$$

$$\mathit{TxID} = H_{512}\big(\mathsf{Enc}(\texttt{PLAU/TX/EFFECTS},\ \mathit{TxCore})\big), \qquad \mathit{AuthMsg} = \mathsf{Enc}(\texttt{PLAU/TX},\ \mathit{TxID}) \tag{7.5}$$

$$\begin{aligned}\mathsf{Conflict}(tx, tx') \iff\ &\text{shared outpoint} \ \vee\ \text{shared } (\mathit{accountID}, \mathit{expectedAccountVersion})\\ &\vee\ \text{shared } \mathit{receiptID} \ \vee\ \text{shared } \mathit{rewardID}\\ &\vee\ \text{shared } \mathit{InitialNonce} \ \vee\ \text{competing } \mathit{InitialValve} \text{ update}\end{aligned} \tag{7.6}$$

Legend:

- *op* — a consumed public outpoint; pk_{op} — the presented key; $\mathit{lockingPolicy}(op)$ — the immutable policy of the output;
- *TxCore* — the canonical effect record binding all inputs, public outputs, amounts, fees and bounded job/lease/L2 metadata;
- $\mathsf{Conflict}$ — the dependency relation that, alongside the scheduler guard (6.15a), serializes validation.

7.3. Transparent Escrow Funding

Every commercial job is funded exclusively from auditable public UTXOs with public change (7.7). There are no shielded inputs, hidden co-inputs or anonymous balance pools, ensuring complete ledger auditability across the job lifecycle.

$$\sum_{o \in inputs} v_o = \sum_{o' \in change} v_{o'} + d_J + f_{\text{queue}}, \qquad \Delta U = -(d_J + f_{\text{queue}}), \qquad \Delta E_{\text{esc}} = d_J, \qquad \Delta R_l = f_{\text{queue}} \tag{7.7}$$

Legend:

- $v_o, v_{o'}$ — the values of consumed public UTXOs and newly created public change outputs;
- d_J — the job escrow deposit; f_{queue} — the queue entry and retention fee (7.16).

7.4. Commercial Trade-Secret Abstraction

While monetary flows ($U \to E_{\text{esc}} \to U$) are public by design, the on-chain job manifest carries raw numerical matrix tiles and execution terms only, decoupling the mathematical workload from off-chain business context (7.8). Point-to-point transport of auxiliary commercial payloads between a client and an assigned executor uses authenticated ML-KEM-1024 / AES-256-GCM envelopes (2.6).

$$\begin{aligned} \mathcal{D}_{\text{pub}} &= \{\text{public funding outpoints, output values/policies, fees, job/lease IDs, raw numeric tiles, dimensions, timing}\} \\ \mathcal{D}_{\text{hidden}} &= \{\text{off-chain business labels, task purpose, unpublished model topology, proprietary context outside the tiles}\} \end{aligned} \tag{7.8}$$

Legend:

- $\mathcal{D}_{\text{pub}}$ — data public on chain and reconstructed by consensus nodes; consensus acceptance depends solely on $\mathcal{D}_{\text{pub}}$ and verified proofs;
- $\mathcal{D}_{\text{hidden}}$ — off-chain business metadata omitted from the protocol manifest; point-to-point envelopes follow (2.6).

7.5. Atomic Transitions and Settlement

Every operation moves value between ten disjoint categories by deltas that sum to zero (7.9), committing as one indivisible database transaction or rolling back completely (7.10). Settlement is the guarded transition (7.11), the lifecycle deltas are (7.12), and a worked job budget is (7.13).

$$\Delta U + \Delta U_{\text{initial}} + \Delta E_{\text{esc}} + \Delta B_{\text{bond}} + \Delta V_{\text{L2}} + \Delta R_a + \Delta R_l + \Delta P_{\text{pending}} + \Delta R_I + \Delta R_M = 0 \tag{7.9}$$

$$\Sigma' = \begin{cases} \Sigma \oplus \Delta & \text{if } \mathsf{Pre}(\Sigma, tx) \wedge \mathsf{Inv}(\Sigma \oplus \Delta), \\ \Sigma & \text{otherwise} \end{cases} \tag{7.10}$$

$$\begin{aligned} \mathsf{Pre}_{\text{settle}} \iff \mathsf{LeaseMatch}(&\mathit{jobID},\ \mathit{tileID},\ \mathit{leaseVersion},\ \mathit{executor}) \ \wedge\ \mathit{status} = \text{Running} \\ &\wedge\ h \le h_{\text{deadline}} \ \wedge\ \mathit{receiptID} \notin \mathcal{R} \ \wedge\ \text{ComputeValid} \ \wedge\ \mathsf{Admit}_{\text{DA}} \\ &\wedge\ E_{\text{esc}} \ge v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}}, \qquad \mathcal{R}' = \mathcal{R} \cup \{\mathit{receiptID}\} \end{aligned} \tag{7.11}$$

$$\begin{aligned} \texttt{ESCROW_FUND_AND_QUEUE}: &\quad \Delta U = -(d_J + f_{\text{queue}}), \quad \Delta E_{\text{esc}} = d_J, \quad \Delta R_l = f_{\text{queue}} \\ \texttt{JOB_ADMIT_AND_LEASE}: &\quad \Delta E_{\text{esc}} = -F^{\text{base}}, \quad \Delta R_l = F^{\text{base}} \\ \texttt{RESULT_SETTLE}: &\quad \Delta E_{\text{esc}} = -(v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}}), \quad \Delta U = v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}} \\ \texttt{ESCROW_REFUND}: &\quad \Delta E_{\text{esc}} = -v_{\text{ref}}, \quad \Delta U = v_{\text{ref}} \\ \texttt{BOND_LOCK / RELEASE}: &\quad U \leftrightarrow B_{\text{bond}} : b, \qquad \texttt{BOND_SLASH}: \ \Delta B_{\text{bond}} = -v_{\text{slash}},\ \Delta R_l = v_{\text{slash}} \\ \texttt{RESERVE_MATURE}: &\quad \Delta R_l = -v_{\text{mat}}, \quad \Delta R_a = v_{\text{mat}} \\ \texttt{FEE_REWARD_ALLOCATE}: &\quad \Delta R_a = -v_{\text{seg}}, \quad \Delta P_{\text{pending}} = v_{\text{seg}} = P_e = \textstyle\sum_i \text{Reward}_{i,e} \le E_e \\ \text{primary block release}: &\quad \Delta R_I = -i_k,\ \Delta U_{\text{initial}} = i_k, \qquad \Delta R_M = -m_k,\ \Delta P_{\text{pending}} = m_k \\ \texttt{FW_REWARD}: &\quad \Delta P_{\text{pending}} = -v_{\text{pay}}, \quad \Delta U = v_{\text{pay}} \\ \texttt{INITIAL_POOL_WITHDRAWAL}: &\quad \Delta U_{\text{initial}} = -(v + f), \quad \Delta U = v, \quad \Delta R_l = f \end{aligned} \tag{7.12}$$

$$d_J = F^{\text{base}} + v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}} = (10 + 80 + 5 + 5)\ \text{PLAU} = 100\ \text{PLAU} \tag{7.13}$$

Legend:

- Δ — the balance change in one operation across the ten categories of (7.15);
- d_J — the job escrow deposit; i_k, m_k — the block-k primary releases from R_I and R_M;
- $v_{\text{exec}}, v_{\text{DA}}, v_{\text{ref}}, v_{\text{slash}}, v_{\text{mat}}, v_{\text{seg}}, v_{\text{pay}}$ — the executor, DA, refund, slash, maturing, segregated and paid amounts;
- a reorganization reverts all monetary deltas, UTXO outpoints, receipts, emission counters and valve buckets jointly.

7.6. Supply Invariant, Decoupled Issuance and InitialProtocolPool

The total supply is fixed at M atoms (7.14). At Genesis all issued balances are zero; 10% of supply resides in the unissued initial reserve $R_I(0) = I$ and 90% in the unissued miner reserve $R_M(0) = R$, while the ten disjoint balances sum to M in every reachable state (7.15)–(7.19a). As blocks $k = h - h_{\text{G}} \geq 1$ are accepted, R_I releases linearly over $N_I = 1{,}051{,}200$ blocks into the protected `InitialProtocolPool` (U_{initial}) (7.19b)–(7.19c), and R_M releases to miner pending rewards via an exact integer Q64 exponential recurrence with half-life $H_M = 1{,}051{,}200$ blocks (7.19d)–(7.19g). Every debit from U_{initial} requires the treasury dual-signature predicate (2.5) and passes through the persistent 720-block rolling gross-outflow valve `InitialValve`, capped at $L_I = I/50 = 30{,}000$ PLAU per window (7.19h)–(7.19j).

Theorem 7.1 (Supply conservation). If Genesis satisfies (7.15) and every accepted block and operation satisfies (7.9) under the atomic commit rule (7.10), then every reachable state satisfies (7.15).

Proof. Genesis satisfies $R_I(0) + R_M(0) = I + R = M$ with all other eight balances equal to zero. Primary releases (7.19c) and (7.19g) move exact integer deltas i_k and m_k from R_I to U_{initial} and from R_M to P_{pending}, preserving the sum. Every accepted operation preserves the balance sum by (7.9) under (7.10), while a rejected operation or block leaves Σ unchanged; induction over accepted blocks and atomic reorganization rollback therefore establishes (7.15).

$$\begin{aligned} &1\ \text{PLAU} = 10^8\ \text{atoms}, \qquad M = 1{,}500{,}000{,}000{,}000{,}000\ \text{atoms} = 15{,}000{,}000\ \text{PLAU} < 2^{51} < p \\ &I = M/10 = 150{,}000{,}000{,}000{,}000\ \text{atoms} = 1{,}500{,}000\ \text{PLAU} \\ &R = 9M/10 = 1{,}350{,}000{,}000{,}000{,}000\ \text{atoms} = 13{,}500{,}000\ \text{PLAU} \end{aligned} \tag{7.14}$$

$$\boxed{U + U_{\text{initial}} + E_{\text{esc}} + B_{\text{bond}} + V_{\text{L2}} + R_a + R_l + P_{\text{pending}} + R_I + R_M = M} \tag{7.15}$$

$$R_I(0) = I, \qquad R_M(0) = R, \qquad U(0) = U_{\text{initial}}(0) = E_{\text{esc}}(0) = B_{\text{bond}}(0) = V_{\text{L2}}(0) = R_a(0) = R_l(0) = P_{\text{pending}}(0) = 0 \tag{7.19a}$$

$$N_I = 1{,}051{,}200, \qquad I_k = \left\lfloor \frac{I \min(k, N_I)}{N_I} \right\rfloor, \qquad i_k = I_k - I_{k-1}, \qquad R_I(k) = I - I_k \tag{7.19b}$$

$$\sum_{k=1}^{N_I} i_k = I, \qquad i_k = 0 \quad (k > N_I), \qquad \Delta U_{\text{initial}} = i_k, \qquad \Delta R_I = -i_k \tag{7.19c}$$

$$I_M^*(k) = R\big(1 - 2^{-k/H_M}\big), \qquad m_k^* = R\big(1 - 2^{-1/H_M}\big)2^{-(k-1)/H_M}, \qquad H_M = 1{,}051{,}200 \tag{7.19d}$$

$$Q = 2^{64}, \qquad a = \big\lfloor Q\, 2^{-1/H_M} \big\rfloor = 18{,}446{,}731{,}910{,}177{,}940{,}463, \qquad S_0 = RQ \tag{7.19e}$$

$$\widehat{S}_k = \left\lfloor \frac{aS_{k-1}}{Q} \right\rfloor, \qquad S_k = \begin{cases} 0 & \widehat{S}_k < Q \\ \widehat{S}_k & \widehat{S}_k \geq Q \end{cases} \tag{7.19f}$$

$$R_M(k) = \big\lfloor S_k/Q \big\rfloor, \qquad I_M(k) = R - R_M(k), \qquad m_k = R_M(k-1) - R_M(k), \qquad \Delta R_M = -m_k, \quad \Delta P_{\text{pending}} = m_k \tag{7.19g}$$

$$W = 720, \qquad L_I = \frac{I}{50} = 3{,}000{,}000{,}000{,}000 \text{ atoms} = 30{,}000 \text{ PLAU}, \qquad \forall\, k \geq 1 : \sum_{t=\max(1,\, k-W+1)}^{k} w_t \leq L_I \tag{7.19h}$$

$$w_k = \sum_{tx \in \mathsf{InitialPoolWithdrawals}(k)} \left(\sum_{o \in \mathsf{publicOutputs}(tx)} v_o + f_{\text{pool}}(tx) \right) \tag{7.19i}$$

$$\mathsf{InitialOK}(tx, \Sigma, k) \iff \mathsf{Accept}_{\text{treasury}}(tx) = 1 \wedge \nu_{\text{expected}} = \mathit{InitialNonce} \wedge U_{\text{initial}} \geq v{+}f \wedge w_{\text{rolling}}{+}v{+}f \leq L_I \tag{7.19j}$$

Legend:

- I, R — the lifetime initial protocol allocation ($10\%M$) and miner emission allocation ($90\%M$); $R_I(k), R_M(k)$ — their unissued balances at block $k = h - h_{\text{G}}$;
- I_k, i_k — cumulative and per-block linear release into U_{initial} over $N_I = 1{,}051{,}200$ accepted blocks;
- $Q = 2^{64}, a, S_k$ — the fixed-point Q64 decay multiplier and sub-atom reserve state; splitting $S = uQ + v$ evaluates $\lfloor aS/Q \rfloor = ua + \lfloor va/Q \rfloor$ in bounded 128-bit integer arithmetic, releasing the final atom when $\widehat{S}_k < Q$;
- $W = 720, L_I, w_k$ — the rolling window length, the 2% gross-outflow cap (30,000 PLAU) and the block-k gross outflow from U_{initial} including pool-funded fees;
- $\mathsf{InitialOK}$ — the withdrawal guard enforcing dual-signature treasury authorization (2.5), replay nonce *InitialNonce* and the rolling valve limit.

7.7. Fee Schedule and Anti-Spam Base Fee

Queue entry and reserved capacity are priced separately, with positive integer tariffs and no tips or ordering auctions (7.16) [37]. At the exact moment of `JOB_ADMIT_AND_LEASE`, the Base Fee transfers irretrievably from the escrow to the locked reserve (7.17). If the client does not provide the data matrix by $h_{\text{admit}} + D_{\text{avail}}$, it forfeits the queue fee and the Base Fee, and the executor bears no penalty (7.18). Queue capacity therefore cannot be held for free.

$$F_J = f_J^{\text{queue}} + F_J^{\text{base}} + F_J^{\text{exec}} + F_J^{\text{DA}}, \qquad f_{\text{queue}} = f_{\text{entry}} + \tau_{\text{byte}}\, B_{\text{queue}}\, H_{\text{TTL}} > 0 \tag{7.16}$$

$$\begin{aligned} \texttt{JOB_ADMIT_AND_LEASE}: \;\; & E_J \leftarrow E_J - F_J^{\text{base}}, \qquad R_l\big[e(h_{\text{admit}}) + L\big] \leftarrow R_l\big[e(h_{\text{admit}}) + L\big] + F_J^{\text{base}} \\ & \forall\, op \in \mathsf{Ops}: \;\; \text{flow}_{op}(R_l \to E_J) = 0 \end{aligned} \tag{7.17}$$

$$\begin{aligned} \neg\, \texttt{INPUT_READY} \text{ by } h_{\text{admit}} + D_{\text{avail}} \;\Rightarrow\; & \text{Loss}_{\text{client}} = f_J^{\text{queue}} + F_J^{\text{base}}, \\ & \text{penalty}_{\text{executor}} = 0, \qquad F_J^{\text{exec}} + F_J^{\text{DA}} \in E_J \end{aligned} \tag{7.18}$$

$$\big|\{op : op.id = \iota\}\big| \leq 1 \quad \text{for } \iota \in \{\mathit{TxID},\ \mathit{ShareID},\ \mathit{admissionID},\ \mathit{receiptID},\ \mathit{rewardID}\} \tag{7.19}$$

Legend:

- F_J — the total charge of job J; $F_J^{\text{exec}}, F_J^{\text{DA}}$ — the executor and DA remuneration, held in escrow until earned;
- f_{entry} — the entry and validation fee; τ_{byte} — the price per byte-block; B_{queue} — the accounted job-state bytes; H_{TTL} — the paid retention in blocks;
- E_J — the escrow of J; $R_l[\cdot]$ — the locked reserve indexed by maturity epoch; $e(h)$ — the epoch containing height h; L — the maturation delay in epochs;
- $\text{flow}_{op}(R_l \to E_J)$ — value moved by op from the locked reserve back to the escrow or the client, which is zero for every operation;
- $\text{Loss}_{\text{client}}$ — the forfeited amount; the unearned remuneration stays in E_J for a paid re-admission or is refunded (6.14);
- ι — an identifier class; repeated identifiers are rejected without state change.

7.8. Recycling and Exact Reserve Dynamics

Fees are recycled, never burned, maturing from R_l into R_a as independent, immutable fee lots c, preventing floor-rounding manipulation. A fee F_c paid in epoch e_c matures in $m_c = e_c + L + 1$. Each lot funds rewards through an exact integer rational rate $\eta = u/v$ ($\gcd(u, v) = 1$) (7.20)–(7.21). Dust from integer flooring is permanently segregated in R_a and never funds later budgets (7.22). Primary Q64 miner release (7.19g) is accounted independently, so no Genesis fee lot is initialized in R_a.

$$\text{public payer} \to R_l \xrightarrow{\text{maturation}} R_a \to \text{independent fee lots } c \to P_{\text{pending}} \to U \tag{7.20}$$

$$\begin{gathered}
W_e = \sum_{s \in \mathcal{S}_e} w_s, \qquad b_{c,e} = \begin{cases} \left\lfloor \frac{uA_{c,e}+v-1}{v} \right\rfloor & W_e > 0 \\ 0 & W_e = 0 \end{cases}, \qquad \text{Reward}_{s,c,e} = \begin{cases} \left\lfloor \frac{b_{c,e} w_s}{W_e} \right\rfloor & s \in \mathcal{S}_e,\ W_e > 0 \\ 0 & \text{otherwise} \end{cases} \\
\text{Reward}_{i,e} = \sum_{c \in \mathcal{C}_e} \sum_{\substack{s \in \mathcal{S}_e \\ \text{owner}(s) = i}} \text{Reward}_{s,c,e}, \qquad p_{c,e} = \sum_{s \in \mathcal{S}_e} \text{Reward}_{s,c,e}, \qquad r_{c,e} = b_{c,e} - p_{c,e} \\
P_e = \sum_{c \in \mathcal{C}_e} p_{c,e}, \qquad E_e = \sum_{c \in \mathcal{C}_e} b_{c,e}, \qquad r_e = \sum_{c \in \mathcal{C}_e} r_{c,e} = E_e - P_e, \qquad \sum_{e \geq m_c} p_{c,e} \leq \sum_{e \geq m_c} b_{c,e} \leq F_c
\end{gathered} \tag{7.21}$$

$$\begin{gathered}
A_{c,e+1} = A_{c,e} - b_{c,e}, \qquad D_{e+1} = D_e + r_e, \qquad G_{e+1} = \sum_{c:\, m_c = e+1} F_c = F_{e-L} \\
R_{a,e} = D_e + \sum_{c \in \mathcal{C}_e} A_{c,e}, \qquad \boxed{R_{a,e+1} = R_{a,e} - E_e + r_e + G_{e+1} = R_{a,e} - P_e + F_{e-L}} \\
u, v \in \mathbb{Z}_{>0}, \quad 0 < u < v, \qquad A_{c,m_c} = F_c, \qquad R_{a,0} = D_0 = 0, \qquad \mathcal{C}_0 = \varnothing
\end{gathered} \tag{7.22}$$

Legend:

- $A_{c,e}$ — unbudgeted principal of fee lot c; D_e — permanently segregated rounding dust; $\mathcal{C}_e$ — active fee lots ($R_{a,0} = D_0 = 0$, $\mathcal{C}_0 = \varnothing$ at Genesis);
- W_e — total mature integer share weight (5.15); $b_{c,e}$ — exact integer ceiling budget; $\text{Reward}_{s,c,e}$ — exact floor entitlement per ShareID; $p_{c,e}$ — booked payouts; $r_{c,e}$ — unpayable dust;
- P_e, E_e, r_e — global booked fee payouts, budget, and dust; G_{e+1} — newly matured fees;
- u, v — numerator and denominator of η; all divisions are exact Euclidean integers.

7.9. Self-Dealing Theorem

A coalition that pays itself for fictitious jobs gains neither weight nor shares, and it recovers strictly less than its fee under exact integer accounting (7.23)–(7.24).

Theorem 7.2 (Self-dealing is loss-making). For every coalition with $\bar{\alpha}\beta^{L+1} < 1$, conditional on future A5 fairness, (7.25) holds even when execution and DA payments are fully recovered and computation is free.

Proof. A new lot c changes no existing lot's budget and is independent of primary Q64 issuance. Flooring ensures the coalition's booked payout $P_{A,c,e} \leq \rho_{A,e} b_{c,e}$. Summing the discounted lifetime returns bounds the recovery at most by $\bar{\alpha}\beta^{L+1}F_c$, yielding strict loss because that bound is strictly below F_c.

$$\rho_{A,e} = \begin{cases} \frac{1}{W_e} \sum_{s \in \mathcal{S}_e \cap A} w_s & W_e > 0 \\ 0 & W_e = 0 \end{cases}, \qquad \bar{\alpha} = \alpha + \delta_f(1 - \alpha), \qquad \rho_{A,e} \leq \bar{\alpha} \ \text{ on } \mathcal{F}_\delta \ (E_e > 0) \tag{7.23}$$

$$PV_{\text{return}} = \sum_{e \geq m_c} \beta^{e - e_c} P_{A,c,e} \leq \sum_{e \geq m_c} \beta^{e - e_c} \bar{\alpha} b_{c,e} \leq \bar{\alpha}\beta^{L+1} F_c \quad \text{on } \mathcal{F}_\delta \tag{7.24}$$

$$\boxed{\Delta\Pi_{\text{self}} \leq -\big(1 - \bar{\alpha}\beta^{L+1}\big)F - C_{\text{res}} - C_{\text{cap}} < 0} \quad \text{on } \mathcal{F}_\delta \text{ if } \bar{\alpha}\beta^{L+1} < 1 \tag{7.25}$$

$$\begin{gathered}
\Pr(\mathcal{F}_\delta^c \mid \mathcal{H}_{e_c}) \leq \varepsilon_{\text{bad}} \quad \text{uniformly}, \qquad \bar{\alpha}_\varepsilon = \bar{\alpha} + \varepsilon_{\text{bad}}(1 - \bar{\alpha}), \qquad \bar{\alpha}_\varepsilon \beta^{L+1} < 1 \\
\boxed{\mathbb{E}[\Delta\Pi_{\text{self}} \mid \mathcal{H}_{e_c}] \leq -\big(1 - \bar{\alpha}_\varepsilon \beta^{L+1}\big)F - \mathbb{E}[C \mid \mathcal{H}_{e_c}] < 0}
\end{gathered} \tag{7.26}$$

Legend:

- $\rho_{A,e}$ — coalition's realized weight fraction; $\bar{\alpha}$ — future payout share bound on $\mathcal{F}_\delta$ for positive-budget epochs $E_e > 0$ (when $E_e = 0$, payouts are identically zero);
- $F = F_c > 0$ — Base Fee of the fictitious job; $\beta \in (0, 1]$ — per-epoch discount factor;
- PV_{return} — present value returned; $C = C_{\text{res}} + C_{\text{cap}} \geq 0$ — resource and capital costs;
- ε_{bad} — bounds fairness failure uniformly under adaptive admission; $\mathcal{H}_{e_c}$ — admission information, with $\bar{\alpha}_\varepsilon \beta^{L+1} < 1$ establishing strict expected loss.

7.10. Resource Limits and Transparent L2 Slots

Every block satisfies a componentwise resource vector that accounts for post-quantum object sizes and compute verification (7.27). Layer 2 attaches only through versioned transparent-value settlement slots whose vaults cover all public claims (7.28) [9], [38]. Activation requires DA rules and an operator-failure exit creating public UTXOs.

$$\sum_{tx \in \mathsf{blk}} \vec{c}(tx) \preceq \big(B_{\text{bytes}},\ B_{\text{sig}},\ B_{\text{compute}},\ B_{\text{state}},\ B_{\text{DA}}\big), \qquad |\sigma_{\mathsf{ML\text{-}DSA\text{-}87}}| = 4627 \text{ B}, \quad |\sigma_{\mathsf{SLH\text{-}DSA}}| = 29{,}792 \text{ B} \tag{7.27}$$

$$\begin{aligned} \mathsf{RollupSlot} = (&\mathit{RID},\ \mathit{verifierID},\ \mathit{programHash},\ \mathit{root},\ \mathit{seq},\\ &\mathit{inboxCursor},\ \mathit{outboxRoot},\ \mathit{vaultBalance},\ \mathit{DApolicy},\ \mathit{exitPolicy}) \\ V_{\mathit{RID}} = &\text{ active public L2 claims} + \text{authorized unpaid exits} + \text{unbooked deposits} \end{aligned} \tag{7.28}$$

Legend:

- $\vec{c}(tx)$ — the resource vector of tx; $\preceq$ — componentwise inequality;
- $B_{\text{bytes}}, B_{\text{sig}}, B_{\text{compute}}, B_{\text{state}}, B_{\text{DA}}$ — the block limits on bytes, signature checks, Freivalds and SciVM compute-proof checks, state writes and DA data;
- RID — the slot identifier; V_{RID} — its transparent vault; an L2 program never writes primary reserves, `InitialProtocolPool`, `InitialValve` or foreign vaults.

References

[1] K. Ozaki, Y. Uchino and T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," arXiv:2504.08009, 2025. arxiv.org/abs/2504.08009.

[2] J. Kepner, P. Aaltonen, D. Bader, A. Buluç, F. Franchetti, J. Gilbert, D. Hutchison, M. Kumar, A. Lumsdaine, H. Meyerhenke, S. McMillan, C. Yang, J. D. Owens, M. Zalewski, T. Mattson and J. Moreira, "Mathematical Foundations of the GraphBLAS," IEEE High Performance Extreme Computing Conference (HPEC), 2016; arXiv:1606.05790. arxiv.org/abs/1606.05790.

[3] ONNX Runtime Documentation, "Quantize ONNX Models: U8S8 intermediate accumulator saturation," Microsoft / LF AI & Data Foundation. onnxruntime.ai/docs/performance/model-optimizations/quantization.html.

[4] M. A. Clark, P. C. La Plante and L. J. Greenhill, "Accelerating Radio Astronomy Cross-Correlation with Graphics Processing Units," International Journal of High Performance Computing Applications, vol. 27, no. 2, 2013, pp. 178–192 (xGPU FX correlator). github.com/GPU-correlators/xGPU.

[5] R. Pass and E. Shi, "FruitChains: A Fair Blockchain," Cryptology ePrint Archive, Paper 2016/916, 2016. eprint.iacr.org/2016/916.

[6] X. Wang, V. V. Muppirala, L. Yang, S. Kannan and P. Viswanath, "Securing Parallel-chain Protocols under Variable Mining Power," arXiv:2105.02927, 2021, §5.2 and Appendix E. arxiv.org/abs/2105.02927.

[7] D. Guo and L. Ren, "Bitcoin's Latency–Security Analysis Made Simple," arXiv:2203.06357, 2022, Theorem 1. arxiv.org/abs/2203.06357.

[8] A.-P. Stouka and T. Zacharias, "On the (De)centralization of FruitChains," arXiv:2307.12422, 2023; DOI 10.1109/CSF57540.2023.00020. arxiv.org/abs/2307.12422.

[9] "Zero-knowledge rollups," Ethereum.org developer documentation. ethereum.org/developers/docs/scaling/zk-rollups.

[10] R. Freivalds, "Probabilistic machines can use less running time," in Information Processing 77, Proceedings of the IFIP Congress, North-Holland, 1977, pp. 839–842.

[11] R. Freivalds, “Fast probabilistic algorithms,” in Mathematical Foundations of Computer Science 1979 (MFCS 1979), Lecture Notes in Computer Science, vol. 74, Springer, 1979. doi.org/10.1007/3-540-09526-8_5.

[12] A. Fiat and A. Shamir, “How to Prove Yourself: Practical Solutions to Identification and Signature Problems,” in Advances in Cryptology — CRYPTO ’86, Lecture Notes in Computer Science, vol. 263, Springer, 1987, pp. 186–194.

[13] E. Ben-Sasson, I. Bentov, Y. Horesh and M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity,” Cryptology ePrint Archive, Paper 2018/046, 2018. eprint.iacr.org/2018/046.

[14] U. Haböck, D. Levit and S. Papini, “Circle STARKs,” Cryptology ePrint Archive, Paper 2024/278, 2024. eprint.iacr.org/2024/278.

[15] E. Ben-Sasson, I. Bentov, Y. Horesh and M. Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” in 45th International Colloquium on Automata, Languages, and Programming (ICALP 2018), LIPIcs, vol. 107, Schloss Dagstuhl, 2018, pp. 14:1–14:17. doi.org/10.4230/LIPIcs.ICALP.2018.14.

[16] E. Ben-Sasson, A. Chiesa and N. Spooner, “Interactive Oracle Proofs,” Cryptology ePrint Archive, Paper 2016/116, 2016; published version: Springer, 2016, pp. 31–60. eprint.iacr.org/2016/116.

[17] A. R. Block, A. Garreta, J. Katz, J. Thaler, P. R. Tiwari and M. Zając, “Fiat-Shamir Security of FRI and Related SNARKs,” ASIACRYPT 2023; Cryptology ePrint Archive, Paper 2023/1071. eprint.iacr.org/2023/1071.

[18] E. Ben-Sasson, “ethSTARK Documentation,” Cryptology ePrint Archive, Paper 2021/582, 2021. eprint.iacr.org/2021/582.

[19] I. S. Reed and G. Solomon, “Polynomial Codes over Certain Finite Fields,” Journal of the Society for Industrial and Applied Mathematics, vol. 8, no. 2, 1960, pp. 300–304. doi.org/10.1137/0108018.

[20] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in Advances in Cryptology — CRYPTO ’87, Lecture Notes in Computer Science, vol. 293, Springer, 1988, pp. 369–378. doi.org/10.1007/3-540-48184-2_32.

[21] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty and S. Saraf, “Proximity Gaps for Reed-Solomon Codes,” FOCS 2020; Cryptology ePrint Archive, Paper 2020/654. eprint.iacr.org/2020/654.

[22] M. Al-Bassam, A. Sonnino and V. Buterin, “Fraud and Data Availability Proofs: Maximising Light Client Security and Scaling Blockchains with Dishonest Majorities,” arXiv:1809.09044, 2018. arxiv.org/abs/1809.09044.

[23] M. Hall-Andersen, M. Simkin and B. Wagner, “Foundations of Data Availability Sampling,” IACR Communications in Cryptology, 2024; Cryptology ePrint Archive, Paper 2023/1079. doi.org/10.62056/a09qudhdj.

[24] C. Dwork and M. Naor, “Pricing via Processing or Combatting Junk Mail,” in Advances in Cryptology — CRYPTO ’92, Lecture Notes in Computer Science, vol. 740, Springer, 1992, pp. 139–147.

[25] M. Ball, A. Rosen, M. Sabin and P. N. Vasudevan, “Proofs of Useful Work,” Cryptology ePrint Archive, Paper 2017/203, 2017; updated version: “Proofs of Work from Worst-Case Assumptions,” CRYPTO 2018, pp. 789–819. eprint.iacr.org/2017/203.

[26] M. Fitzi, A. Kiayias, G. Panagiotakos and A. Russell, “Ofelimos: Combinatorial Optimization via Proof-of-Useful-Work. A Provably Secure Blockchain Protocol,” Cryptology ePrint Archive, Paper 2021/1379, 2021. eprint.iacr.org/2021/1379.

[27] I. Komargodski and O. Weinstein, “Proofs of Useful Work from Arbitrary Matrix Multiplication,” arXiv:2504.09971, version v4 of 13 November 2025; version v1 also with I. Schen. arxiv.org/abs/2504.09971.

[28] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. bitcoin.org/bitcoin.pdf.

[29] J. A. Garay, A. Kiayias and N. Leonardos, “The Bitcoin Backbone Protocol: Analysis and Applications,” EUROCRYPT 2015; Journal of the ACM, vol. 71, no. 4, 2024, pp. 25:1–25:49; Cryptology ePrint Archive, Paper 2014/765. eprint.iacr.org/2014/765.

[30] J. A. Garay, A. Kiayias and N. Leonardos, “The Bitcoin Backbone Protocol with Chains of Variable Difficulty,” CRYPTO 2017; Cryptology ePrint Archive, Paper 2016/1048. eprint.iacr.org/2016/1048.

[31] C. Decker and R. Wattenhofer, “Information Propagation in the Bitcoin Network,” in 13th IEEE International Conference on Peer-to-Peer Computing (P2P), Trento, 2013.

[32] R. Pass, L. Seeman and a. shelat, “Analysis of the Blockchain Protocol in Asynchronous Networks,” Cryptology ePrint Archive, Paper 2016/454, 2016. eprint.iacr.org/2016/454.

[33] A. Dembo, S. Kannan, E. N. Tas, D. Tse, P. Viswanath, X. Wang and O. Zeitouni, “Everything is a Race and Nakamoto Always Wins,” arXiv:2005.10484, 2020. arxiv.org/abs/2005.10484.

[34] P. Gaži, A. Kiayias and A. Russell, “Tight Consistency Bounds for Bitcoin,” ACM CCS 2020; Cryptology ePrint Archive, Paper 2020/661. eprint.iacr.org/2020/661.

[35] I. Eyal and E. G. Sirer, “Majority is not Enough: Bitcoin Mining is Vulnerable,” arXiv:1311.0243, 2013. arxiv.org/abs/1311.0243.

[36] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach and A. Juels, “Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges,” arXiv:1904.05234, 2019. arxiv.org/abs/1904.05234.

[37] T. Roughgarden, “Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559,” arXiv:2012.00854, 2020. arxiv.org/abs/2012.00854.

[38] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry and A. Gervais, “SoK: Layer-Two Blockchain Protocols,” Financial Cryptography and Data Security 2020; Cryptology ePrint Archive, Paper 2019/360. eprint.iacr.org/2019/360.

[39] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, August 2024. csrc.nist.gov/pubs/fips/203/final.

[40] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, August 2024. csrc.nist.gov/pubs/fips/204/final.

[41] National Institute of Standards and Technology, “Stateless Hash-Based Digital Signature Standard,” FIPS 205, August 2024. csrc.nist.gov/pubs/fips/205/final.

[42] National Institute of Standards and Technology, “Recommendations for Key-Encapsulation Mechanisms,” SP 800-227, September 2025. csrc.nist.gov/pubs/sp/800/227/final.

[43] National Institute of Standards and Technology, “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS 202, August 2015. csrc.nist.gov/pubs/fips/202/final.

[44] National Institute of Standards and Technology, “SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash and ParallelHash,” SP 800-185, December 2016. csrc.nist.gov/pubs/sp/800/185/final.

[45] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, May 2010. rfc-editor.org/rfc/rfc5869.

[46] National Institute of Standards and Technology, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” SP 800-38D, November 2007. csrc.nist.gov/pubs/sp/800/38/d/final.

[47] National Institute of Standards and Technology, “Post-Quantum Cryptography” project. csrc.nist.gov/projects/post-quantum-cryptography.

[48] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC 1996), 1996, pp. 212–219. arxiv.org/abs/quant-ph/9605043.

[49] G. Brassard, P. Høyer and A. Tapp, “Quantum Algorithm for the Collision Problem,” arXiv:quant-ph/9705002, 1997. arxiv.org/abs/quant-ph/9705002.

[50] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” SIAM Journal on Computing, vol. 26, no. 5, 1997, pp. 1484–1509.

[51] J. Don, S. Fehr and C. Majenz, “The Measure-and-Reprogram Technique 2.0: Multi-Round Fiat-Shamir and More,” arXiv:2003.05207, 2020. arxiv.org/abs/2003.05207.

[52] M. O. Rabin, “Probabilistic Algorithms in Finite Fields,” SIAM Journal on Computing, vol. 9, no. 2, 1980, pp. 273–280.

[53] A. R. Block and P. R. Tiwari, “On the Concrete Security of Non-interactive FRI,” Cryptology ePrint Archive, Paper 2024/1161, 2024. eprint.iacr.org/2024/1161.

[54] M. L. Balinski and H. P. Young, “Fair Representation: Meeting the Ideal of One Man, One Vote,” Yale University Press, 1982; 2nd ed., Brookings Institution Press, 2001.